MACHINE LEARNING INTERNSHIP

Combined Project Report

Six Computer-Vision and Audio Machine-Learning Systems

Design, Implementation, Results and Honest Limitations

Systems covered in this report:

1. Long-Hair Gender Identification (CNN + ResNet18)
2. Senior-Citizen Identification for Retail (real-time multi-person)
3. Nationality Detection (ethnicity, emotion, age, dress colour)
4. Age & Emotion Detection through Voice (classical → deep learning)
5. Car Colour Detection & Traffic Analysis (detection, colour, pedestrians)
6. Sign-Language Detection (baseline CNN vs. landmark model)

Built with TensorFlow · PyTorch · scikit-learn · OpenCV · MediaPipe · DeepFace · Streamlit

Contents

Figures are numbered continuously (Figure 1 – 42) across all six parts.

Introduction

This report brings together six machine-learning systems built over the course of the internship. Although each was set as a separate task, they share a common shape, and reading them side by side makes that shape clear. Every task arrived as a short, deceptively simple sentence — “identify gender but flip it by hair length,” “flag senior citizens from a camera,” “detect nationality from a photo,” “find a person’s age from their voice,” “detect and colour-code cars at a traffic signal,” “recognise a few signs on a webcam.” In every case the real work turned out to be the same: seeing that the one-line brief was actually several independent sub-problems stacked together, choosing the right tool for each, and wiring them into a pipeline with a small, readable piece of decision logic at the end.

A few themes run through all six projects and are worth stating once, up front, rather than repeating in each part:

- **Pipelines over single models.** No task was solved by one network. Each was decomposed into clear sub-tasks — often two or three models plus a short combining rule — because that structure is what the briefs actually required and what made the systems easy to reason about and verify by hand.
- **Honest separation of self-built and integrated parts.** Where a component was trained from scratch it is called that; where a pretrained model or library (DeepFace, wav2vec2, MobileNet-SSD, MediaPipe) was integrated, that is stated plainly. Being accurate about which parts are original is treated as part of the engineering, not an afterthought.
- **A working interface for every task.** Each system is wrapped in a Streamlit application so it can be demonstrated end to end without touching the command line. Streamlit was chosen throughout because it turns a plain Python script into a clean web app with very little extra code, keeping the focus on the models.
- **Limitations recorded, not hidden.** Every part ends by naming where the system breaks — small hand-labelled datasets, elderly-voice underestimation, occlusion, single-signer training data — because understanding those limits is as important as the numbers.

The tasks also build on one another. The age-and-gender model trained in Part 1 is reused unchanged in Parts 2 and 3, and the automatic face-detection step that Part 1 lists as future work becomes the opening stage of Part 2. Where those links occur they are pointed out, so the six parts read as one connected body of work rather than six isolated exercises.

Each part below follows the same arc: the problem statement, how it was broken down, the components and datasets, the results with figures, the challenges faced, and a short conclusion. Figures are numbered continuously across the whole report.

Part 1 — Long-Hair Gender Identification System

A multi-model approach combining age estimation, gender classification and hair-length detection. Built with TensorFlow, PyTorch and Streamlit.

1.1 Problem Statement

The task is to build a gender-identification feature with an unusual twist in its logic. Instead of simply predicting a person's true gender, the system has to follow a rule that depends on the person's age and hair length.

For people aged between 20 and 30, the system should classify anyone with long hair as female — even if they are actually male — and anyone with short hair as male, even if they are actually female. In this age band, hair length overrides true gender. For people outside this range (under 20 or over 30), the system should ignore hair length completely and simply predict the person's real gender.

The brief explicitly asks for a self-built machine-learning model and a working graphical interface, and states that evaluation is based on the overall behaviour of the model and the functionality of the interface rather than raw accuracy alone. That makes it as much a logic-building exercise as a modelling one: the core challenge is figuring out which sub-problems need solving and how to wire them together correctly.

1.2 Understanding the Problem

On the surface this looks like a gender-classification problem, but reading it carefully shows it is really three problems stacked on top of each other. To produce the required output for any face, the system needs three independent facts about the person:

- **Their approximate age**, so it can decide which rule applies (the 20–30 rule or the normal-gender rule).
- **Their hair length**, which becomes the deciding factor inside the 20–30 band.
- **Their actual gender**, which is the answer used everywhere outside the 20–30 band.

This means no single model can solve the task. The solution has to be a pipeline: one model predicts age and gender, a second predicts hair length, and a small piece of decision logic combines the three predictions. Identifying this structure early was the most important step in the whole project, because it turned a confusing single requirement into three clear, solvable sub-tasks. The final rule can be written in plain language as:

```
if 20 <= age <= 30:
    gender = 'Female' if hair == 'long' else 'Male'
else:
    gender = predicted_real_gender
```

Everything else in this part — the two neural networks, the datasets, the interface — exists only to supply the three values that feed into this rule.

1.3 System Overview

The system is made up of two trained models and one decision layer, all wrapped inside a single web interface. When a user uploads a photo, the image is sent through both models and their outputs are merged by the decision logic before anything is shown on screen.

Component	Framework	What it produces
Age + Gender model	TensorFlow / Keras	Estimated age (number) and real gender (male / female)

Component	Framework	What it produces
Hair-length model	PyTorch (ResNet18)	Hair length (long / short) with a confidence score
Decision logic	Plain Python	Applies the 20–30 rule and outputs the final gender label
GUI	Streamlit	Lets the user upload an image and view all results

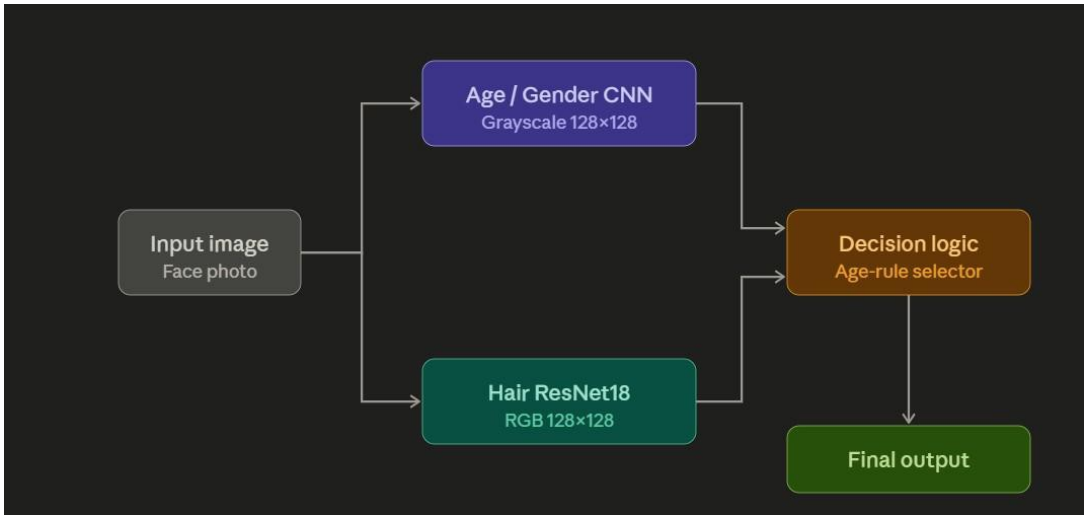


Figure 1 — Task 1 pipeline — the input face is sent to the Age/Gender CNN and the Hair ResNet18 in parallel, and their outputs feed the age-rule decision logic that produces the final label.

1.4 Datasets Used

UTKFace

UTKFace is a large public face dataset of more than twenty thousand images. Its most useful feature is that the age, gender and ethnicity of each person are stored directly inside the filename, separated by underscores — a file named 25_0_2_xxxx.jpg means a 25-year-old male. This made loading labels extremely simple, because they could be parsed straight from the filenames without any separate annotation file. The loader loops over every file, splits on the underscore, and pulls the age and gender from the first two fields:

```
for filename in os.listdir(BASE_DIR):
    temp   = filename.split('_')
    age    = int(temp[0])
    gender = int(temp[1])
    image_paths.append(os.path.join(BASE_DIR, filename))
    age_labels.append(age)
    gender_labels.append(gender)
```

These three lists were placed into a pandas DataFrame so the data could be inspected easily. UTKFace was used both to train the age-and-gender model and as the image source for building the hair-length dataset.

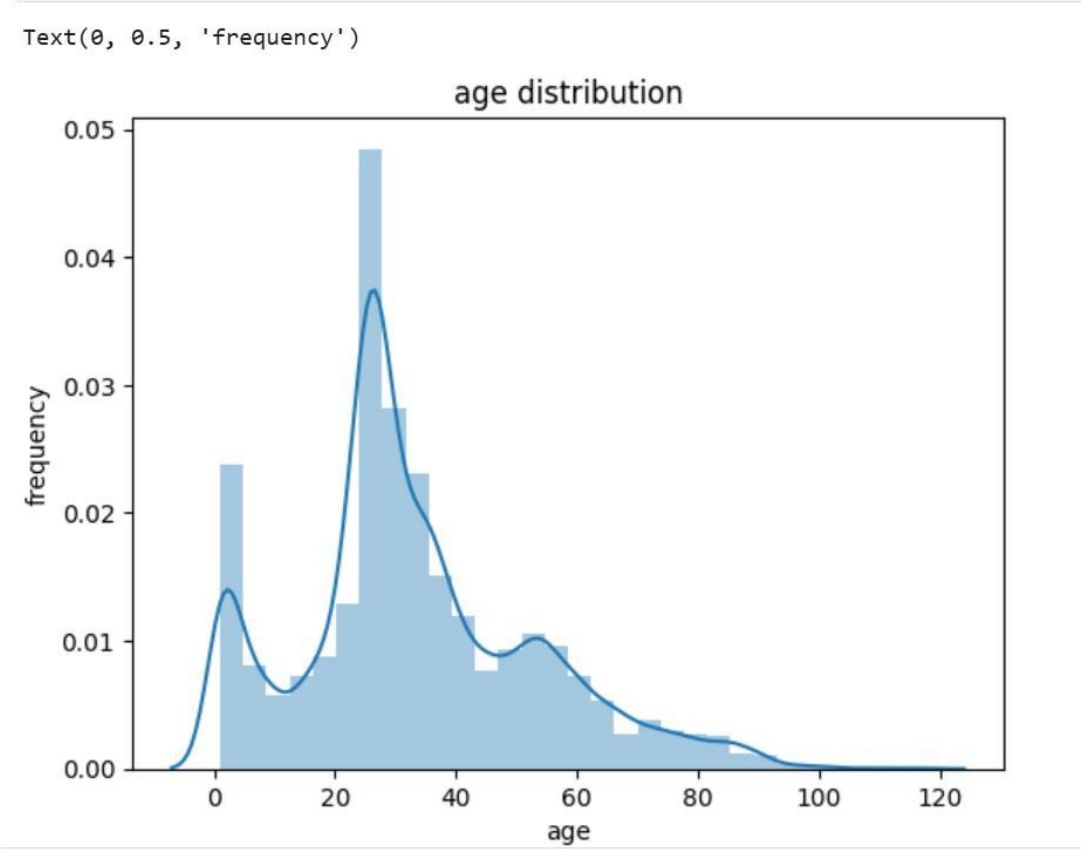
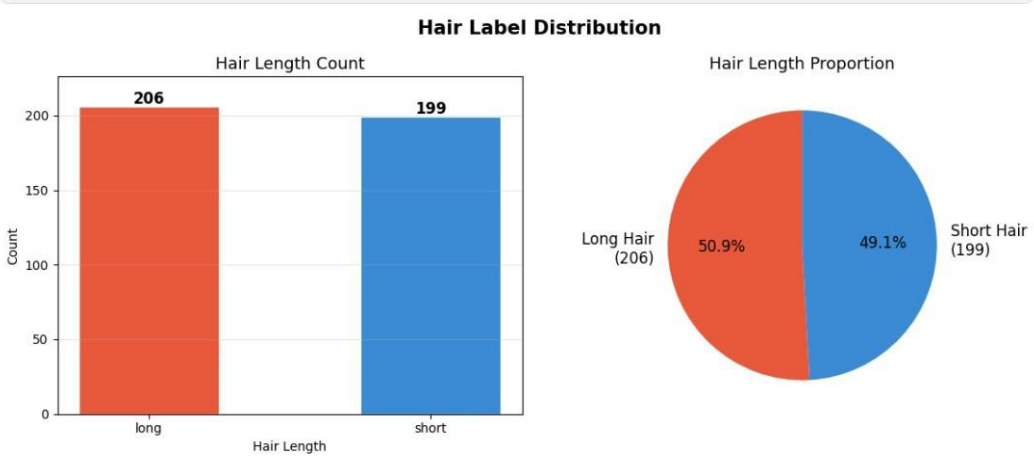


Figure 2 — Age distribution of the UTKFace dataset, concentrated in the 20–40 range.

The hand-built hair dataset

UTKFace contains no information about hair length, so this dataset was one I built and labelled entirely by hand. I sampled five hundred images from the 20–30 age range — balanced as 250 males and 250 females — exported them from the notebook as a zip file, and then went through every single image myself, swiping through the faces one at a time in a small tagging app and marking each as long, short, or skip. The skip option covered images where hair length genuinely could not be judged, such as people wearing hats or photographed from an angle that hid their hair. After removing the skipped ones, around 405 clean, hand-labelled images remained, which I saved to a CSV file and re-uploaded to Kaggle as a private dataset.

I deliberately concentrated this manual effort on the 20–30 band, because that is the only range where hair length actually matters for the final decision — spending the hand-annotation exactly where the problem needs it was a conscious way to get the most value from a small amount of tedious labelling. This hand-labelled set of 405 images became the trusted core of the hair dataset; later (Section 1.6) I expanded it further with confident automatic labels for ages outside the 20–30 band, but these manually assigned labels always remained the ground truth for the difficult middle range.




```
✓ Saved as hair_label_distribution.png
```

Figure 3 — Distribution of the hand-labelled hair dataset — 206 long versus 199 short, a near-even split.

1.5 Task 1 Model — Age and Gender (CNN)

Age and gender are both visual properties of a face, so a Convolutional Neural Network is the natural choice. A CNN slides small filters across the image to detect simple patterns like edges and curves in its early layers, then combines these into more complex features such as eyes, jaw shape and skin texture deeper in the network — exactly the features that distinguish a young face from an old one, or a male face from a female one.

Rather than building two separate networks, one model was designed with a shared body and two output heads. The convolutional layers are shared, and near the end the network splits into a gender branch and an age branch. This multi-task design is efficient: the shared layers learn general facial features useful for both tasks at once, which usually trains faster and generalises better than two isolated models. The model was built with the Keras functional API as four convolutional blocks, each followed by max-pooling, then a flatten and two dense branches:

```
inputs = Input((128, 128, 1))
conv1 = Conv2D(32,  (3,3), activation='relu')(inputs)
maxp1 = MaxPooling2D((2,2))(conv1)
conv2 = Conv2D(64,  (3,3), activation='relu')(maxp1)
maxp2 = MaxPooling2D((2,2))(conv2)
conv3 = Conv2D(128, (3,3), activation='relu')(maxp2)
maxp3 = MaxPooling2D((2,2))(conv3)
conv4 = Conv2D(256, (3,3), activation='relu')(maxp3)
maxp4 = MaxPooling2D((2,2))(conv4)

flatten = Flatten()(maxp4)
dense1  = Dense(256, activation='relu')(flatten)   # gender branch
dense2  = Dense(256, activation='relu')(flatten)   # age branch
drop1   = Dropout(0.3)(dense1)
drop2   = Dropout(0.3)(dense2)

gender_out = Dense(1, activation='sigmoid', name='gender_out')(drop1)
age_out    = Dense(1, activation='relu',    name='age_out')(drop2)
model = Model(inputs=inputs, outputs=[gender_out, age_out])
```

The key layers each play a clear role. The Conv2D layers apply learnable filters whose count grows through the network (32, 64, 128, 256), letting deeper layers detect increasingly abstract features. Max-pooling shrinks each feature map by keeping the strongest activation in every 2×2 region, reducing computation and making the network less sensitive to exact feature position. ReLU introduces the non-linearity that lets the network learn complex relationships. Flatten unrolls the final 3-D feature block into a vector for the dense layers, and Dropout (0.3) randomly disables 30 percent of neurons during training to reduce overfitting. Finally, the gender output uses a sigmoid (a probability between 0 and 1), while the age output uses ReLU to produce a non-negative continuous number.

Preprocessing

Before training, every image was converted to greyscale, resized to 128×128 pixels, and stacked into a single array. Greyscale was deliberate: age and gender are determined mostly by the structure and shape of a face rather than its colour, so dropping colour cuts the input size to a third without losing much useful information. Pixel values were then divided by 255 to scale them into the 0–1 range for more stable training.

```
def extract_feature(images):
    features = []
    for image in images:
        img = Image.open(image).convert('L')   # greyscale
        img = img.resize((128, 128))
        features.append(np.array(img))
    features = np.array(features)
    return features.reshape(len(features), 128, 128, 1)

x = extract_feature(df['Image']) / 255.0
```

Training and results

The data was split 80 / 10 / 10 into training, validation and test. Because the model has two outputs, it was compiled with two losses — binary cross-entropy for the gender classification and mean absolute error for the age regression — using the Adam optimiser, with a scheduler that reduced the learning rate by ten percent each epoch so the model could take smaller, more careful steps as it converged. A ModelCheckpoint callback watched validation gender accuracy and saved only the best-performing weights as best_model.keras, so the final file represents the model at its peak rather than at the last epoch.

```
model.compile(loss=['binary_crossentropy', 'mae'],
              optimizer='adam', metrics=['accuracy', 'mae'])

annealer = LearningRateScheduler(lambda e: 1e-3 * 0.9 ** e)
history = model.fit(X_train, [y_train_gender, y_train_age],
                    batch_size=128, epochs=30,
                    validation_data=(X_val, [y_val_gender, y_val_age]),
                    callbacks=[checkpoint])
```

Over thirty epochs the gender accuracy climbed and levelled off near 0.90 on validation (around 0.94 on training), while the age error steadily fell to roughly six years of mean absolute error.

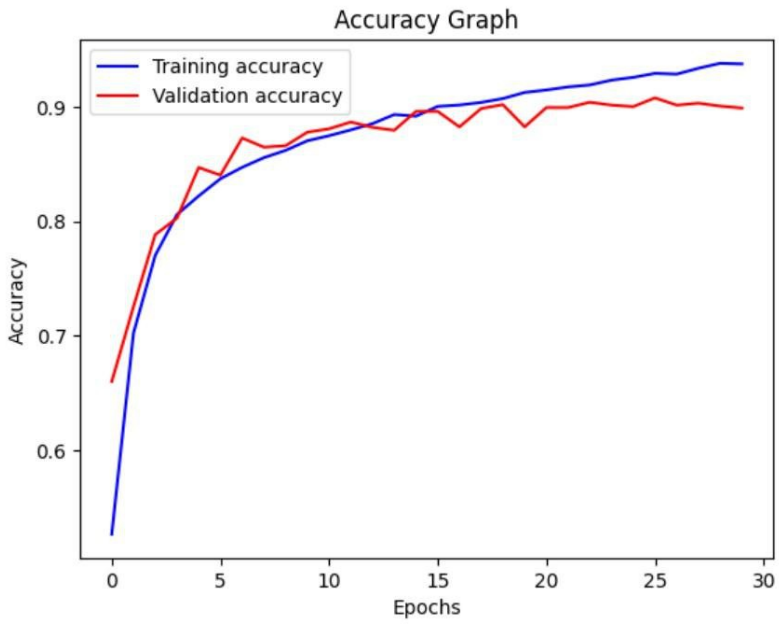


Figure 4 — Gender-branch accuracy over 30 epochs — training and validation both climb and plateau near 0.90.

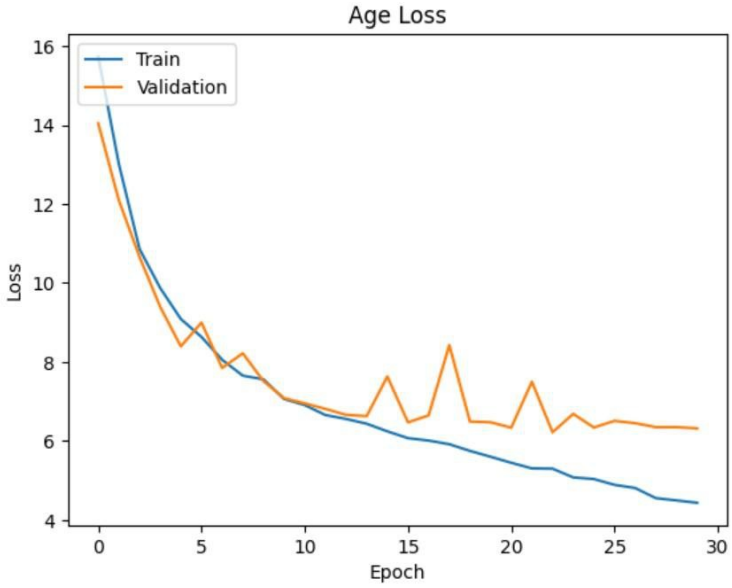


Figure 5 — Age-branch loss (mean absolute error) over training, settling around six years.

1.6 Task 2 Model — Hair Length (ResNet18)

First attempt: rule-based labels from CelebA

The first idea for getting hair-length labels was to avoid manual work entirely by using CelebA, which ships with attribute flags such as Bangs, Wavy_Hair, Straight_Hair, Bald and Male. The plan was to combine these flags with simple rules: bald implies short hair, bangs imply long hair, a female with wavy hair is long, and so on.

```
def get_confident_label(row):
    if row['Bangs'] == 1:                          return 'long'
    if row['Male'] == 0 and row['Wavy_Hair'] == 1: return 'long'
```

```
    if row['Bald'] == 1:                              return 'short'
    if row['Male'] == 1 and row['Bald'] == 0 and \
       row['Bangs'] == 0 and row['Wavy_Hair'] == 0: return 'short'
    return 'unknown'
```

This worked for most cases but had a serious blind spot: CelebA's attributes describe hair texture, not length, so they cannot distinguish a man with a short straight haircut from a man with long straight hair down to his shoulders — both end up labelled short. The model trained on these labels learned a false rule, that straight-haired men are always short-haired. The flaw showed up dramatically in testing, where a man with clearly long straight hair was classified as short with 99.93 percent confidence. Because no amount of tuning can fix bad labels, the attribute-based approach was dropped entirely in favour of the manual UTKFace labels described above.

Expanding the dataset with confident auto-labels

The 405 manual labels covered only the 20–30 range. To give the hair model more data, images outside that range were auto-labelled with a safe heuristic: for people under 20 or over 30, gender is a far more reliable indicator of hair length in this dataset, so females were labelled long and males short. These auto-labels were combined with the hand-made ones, but the manual labels remained the trusted ground truth for the difficult middle range.

```
for f in os.listdir(UTK_DIR):
    age, gender = int(f.split('_')[0]), int(f.split('_')[1])
    if age < 20 or age > 30:                # only auto-label outside 20-30
        label = 'long' if gender == 1 else 'short'
        auto_labels.append({'filename': f, 'hair_length': label})
```

Why ResNet18 and transfer learning

With only a few hundred labelled images, training a deep network from scratch would overfit almost immediately. The solution was transfer learning: starting from a ResNet18 already trained on ImageNet, which knows how to recognise general visual features like edges, textures and shapes, so it only needs nudging toward the specific task of judging hair length. ResNet18's key innovation is the residual (skip) connection, where a block's input is added directly to its output, giving the training signal a shortcut that keeps deep networks trainable and stable. Only the final classification layer was replaced, to output two classes:

```
from torchvision.models import resnet18, ResNet18_Weights
model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(model.fc.in_features, 2)   # long vs short
```

Heavy data augmentation

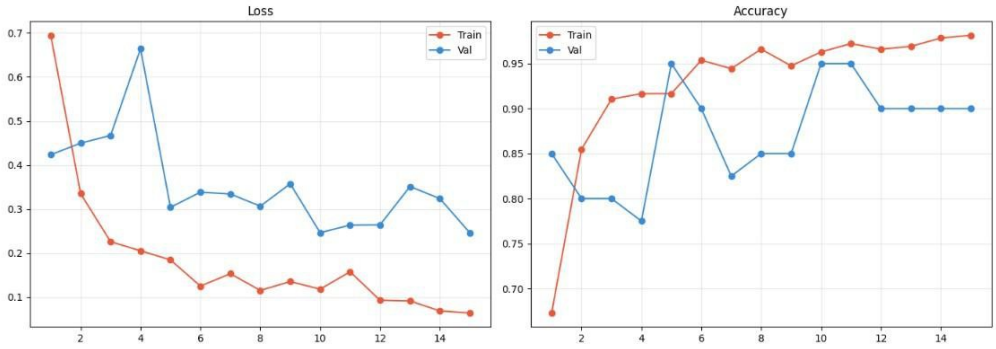
To make the small dataset behave like a larger one, strong augmentation was applied during training so the model effectively never saw the exact same picture twice — horizontal flips, colour jitter, small rotations, occasional blur and random crops. This is one of the most effective ways to fight overfitting when labelled data is limited.

```
train_transform = transforms.Compose([
    transforms.Resize((160, 160)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(0.3, 0.3, 0.3),
    transforms.RandomRotation(10),
```

```
    transforms.RandomApply([transforms.GaussianBlur(3)], p=0.2),
    transforms.RandomResizedCrop(128, scale=(0.8, 1.0)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3),
])
```

Training and results

The model was trained for fifteen epochs with Adam at a learning rate of 0.0001, cross-entropy loss, and a scheduler that halved the learning rate every five epochs. On the held-out test set it reached about 95 percent accuracy with a weighted F1 of 0.95 — 39 of 41 test faces correct — with every short-hair example correct and only two long-hair examples missed.



	precision	recall	f1-score	support
Short hair	0.91	1.00	0.95	20
Long hair	1.00	0.90	0.95	21
accuracy			0.95	41
macro avg	0.95	0.95	0.95	41
weighted avg	0.96	0.95	0.95	41

Figure 6 — Hair model — training and validation loss/accuracy curves with the per-class classification report (precision, recall and F1 around 0.95).

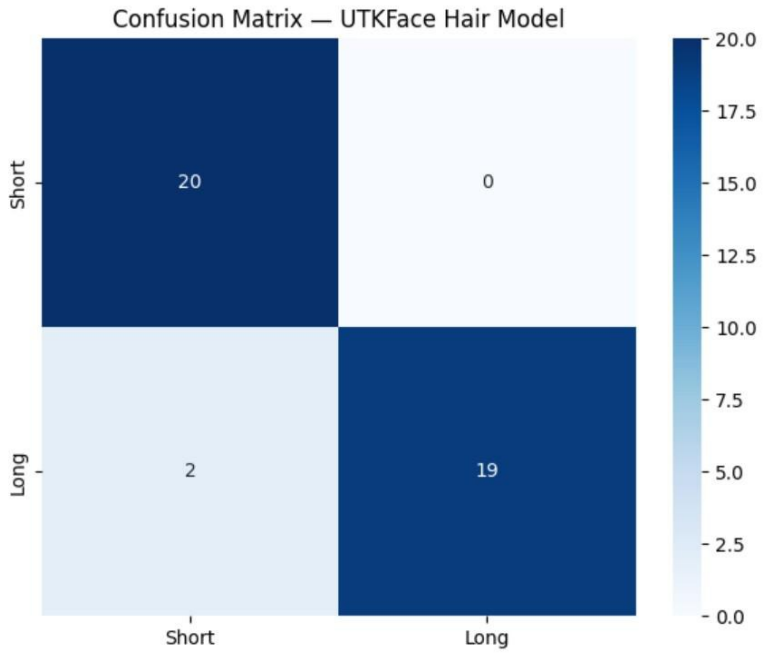


Figure 7 — Hair model confusion matrix on the test set: all 20 short-hair faces correct, 19 of 21 long-hair faces correct.

1.7 The Combined Decision Logic

This short function is where the problem statement is actually answered. The two models supply three values — age, real gender and hair length — and the rule combines them exactly as the task asks: inside the 20–30 band hair length decides the label; outside it, the real predicted gender is used.

```
def final_gender(age, real_gender, hair_length):
    if 20 <= age <= 30:
        # hair length overrides true gender in this band
        return 'Female' if hair_length == 'long' else 'Male'
    else:
        # outside the band, report the real predicted gender
        return real_gender
```

Each branch has a clear purpose. The age check decides which world we are in: between 20 and 30 the task demands that appearance wins over biology, so a long-haired man is reported female and a short-haired woman male; at any other age the true gender is reported and hair is ignored. The two models only supply inputs; this short function satisfies the requirement, and being simple enough to verify by hand matters because the brief weights correct logic as heavily as accuracy.

1.8 The Streamlit Interface

The required GUI was built with Streamlit because it turns a Python script into a clean web app with very little code, keeping the focus on the models. The interface lets a user upload any face photo and instantly see the age, the hair length, and the final gender decision. When an image is uploaded the app prepares two versions of it, because the two models expect different inputs: the age-and-gender model needs a greyscale 128×128 image, while the hair model needs a normalised RGB 128×128 image. Both models run once and their outputs are passed into the decision function.

```
# 1. Age + gender (Keras)
```

```
g_input = preprocess_grey(image)             # greyscale 128x128
gender_p, age_p = keras_model.predict(g_input)
real_gender = 'Female' if round(gender_p[0][0]) == 1 else 'Male'
age = round(age_p[0][0])

# 2. Hair length (PyTorch ResNet18)
h_input = preprocess_rgb(image)              # RGB 128x128 tensor
logits  = hair_model(h_input)
hair    = 'long' if logits.argmax() == 1 else 'short'

# 3. Combine using the task rule
result = final_gender(age, real_gender, hair)
```

Loading two models from two different frameworks in one app is handled by loading each once at start-up and caching it, so uploading a new image does not reload the models. Each prediction completes in well under a second. The layout shows the uploaded image on one side and, on the other, the predicted gender, the estimated age, and the detected hair length with a confidence percentage from the hair model's softmax output.

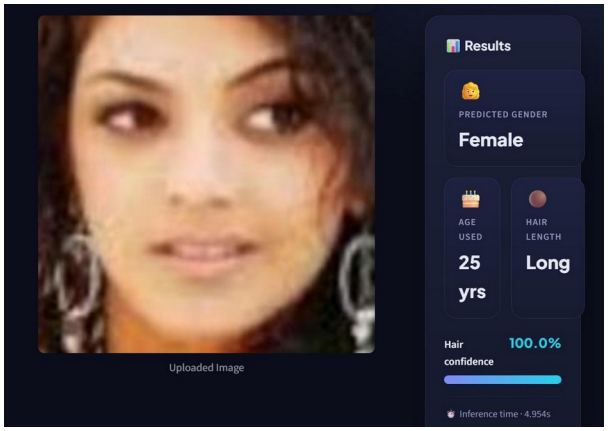


Figure 8 — Interface on a 25-year-old woman with long hair: predicted Female, hair confidence 100%.

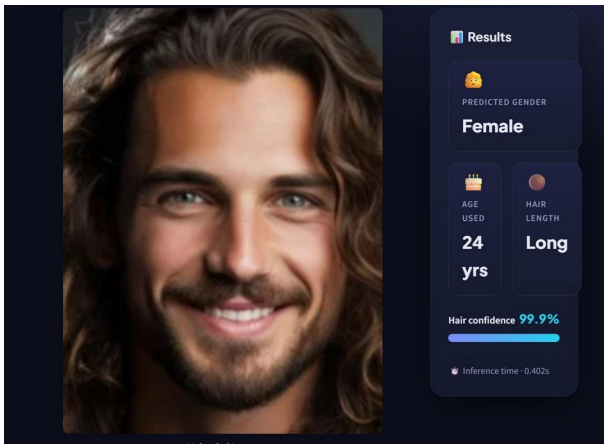


Figure 9 — The rule in action — a 24-year-old man with long hair is reported Female, because inside the 20–30 band hair length overrides true gender.

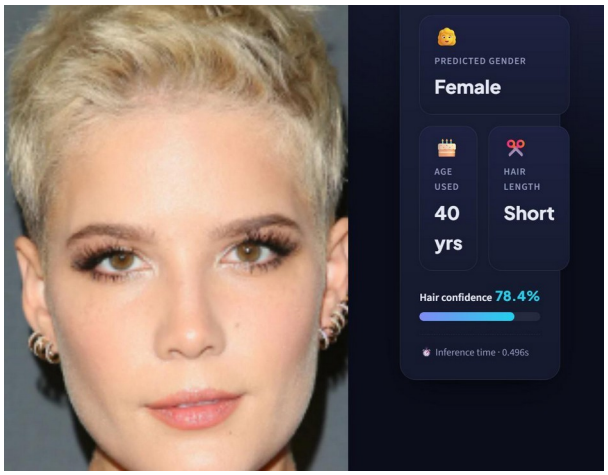


Figure 10 — A 40-year-old woman with short hair: outside the 20–30 band, so hair length is ignored and the real gender is reported.

1.9 Challenges, Results and Conclusion

The biggest challenge was the CelebA labels, which produced confidently wrong labels for long-haired men because the attributes describe texture, not length; this was solved by switching to trusted manual labelling. The second was the small dataset itself: because I hand-labelled all 405 images one at a time, scaling it up to the thousands a fully reliable classifier would need was simply not feasible within the project timeframe, so the set stays small and imbalanced — enough that the model leans on shortcuts like "long hair = female" and is inconsistent on edge cases such as long-haired men and short-haired women. This is recorded as a known limitation: the fix is a larger, balanced dataset, but hand-collecting it at that scale was beyond what was realistic here. Running two frameworks in one app was handled by caching each model at start-up, and the two preprocessing pipelines (greyscale versus normalised RGB) were kept strictly separate with explicit conversions, since mixing them silently produces wrong predictions.

Model	Task	Metric / Result
Custom CNN (Keras)	Age + Gender	Gender accuracy ≈ 90% (val; ≈ 94% train); age MAE ≈ 6.3 years (val)
ResNet18 (PyTorch)	Hair length	Test accuracy ≈ 95%, weighted F1 ≈ 0.95 (39 / 41 correct)
Combined system	Final gender per rule	Correct behaviour verified through the Streamlit GUI

The lesson from this part is that a deceptively simple requirement — identify gender, but flip it by hair length for a specific age band — is really a small system-design problem. Recognising that it needed three predictions rather than one, and that the real work was combining them correctly, was the key insight. The first hair-labelling approach also failed not because of the model but because of biased labels, a reminder that what a model learns is only ever as good as the data it is shown. Natural next steps are a larger hand-labelled hair dataset, more modern backbones such as EfficientNet, and an automatic face-detection step so the app can handle full photographs rather than pre-cropped faces — which is exactly where Part 2 begins.

Part 2 — Senior-Citizen Identification System

Real-time multi-person age, gender and visit logging for retail spaces. Built on the Part 1 age-and-gender model (best_model.keras), with OpenCV and a Streamlit dashboard.

2.1 Problem Statement

The second task moves from a single cropped face to a live retail environment. The system watches a video stream or webcam feed from a mall or store, finds every person in view, and estimates each one's age and gender. Anyone whose estimated age is 60 or above is flagged as a senior citizen. For every logged person, the system records their age, gender and time of visit into a CSV file, so the store ends up with a running log of who walked in and when. A graphical interface is optional here, but a full one was built anyway; the emphasis is on a working model and correct, reliable logging.

2.2 Understanding the Problem

Where Part 1 received one neatly cropped face, Part 2 has to do three new things on top of the age-and-gender prediction it already knows how to do. First, it must locate faces, because a camera frame in a store contains a whole scene — and that scene may hold several people at once. Second, it must apply a single rule to each face: is the estimated age 60 or above? Third, it must persist its findings to disk in a structured file, so the output is no longer flashed on a screen and forgotten but a permanent, timestamped record.

Read this way, the age-and-gender prediction is the part that is already solved. The Part 1 model, best_model.keras, was trained on UTKFace — which contains faces across the full age range, including people well over 60 — so it can be reused unchanged. The genuinely new engineering is everything around it: detecting multiple faces per frame, steadying the predictions, deciding the senior flag, avoiding logging the same person on every frame, and writing clean rows to a file. This also closes a loop left open in Part 1, whose future-work section noted that an automatic face-detection step would let the app handle full photographs rather than pre-cropped faces. Part 2 is essentially that step, taken all the way to a live, multi-person stream with a dashboard on top.

2.3 System Overview

The system is a short pipeline wrapped around the reused model and presented through a Streamlit dashboard. A camera frame goes first to the face detector, which returns a box for each face; every box is cropped and sent through the age-and-gender model; a light tracker links boxes across frames so the same person keeps one identity; a short buffer smooths each person's predictions over several frames; the senior rule is applied; and the first time a person is confirmed, their age, gender and visit time are appended to the log file. The annotated frame, live statistics and the growing log are all shown in the browser.

Component	Framework	What it produces
Age + Gender model	TensorFlow / Keras (best_model.keras, reused from Part 1)	Estimated age and gender for one face
Face detector	OpenCV (Haar cascade)	A bounding box for every face found in a frame
Tracker + smoothing buffer	Plain Python	A stable ID per person and a steadied age/gender over 10 frames
Senior + logging logic	Plain Python	Applies the age ≥ 60 rule and writes a timestamped row to the CSV

Component	Framework	What it produces
Graphical interface	Streamlit	Webcam / upload modes, live stats, and the visit-log viewer with CSV download

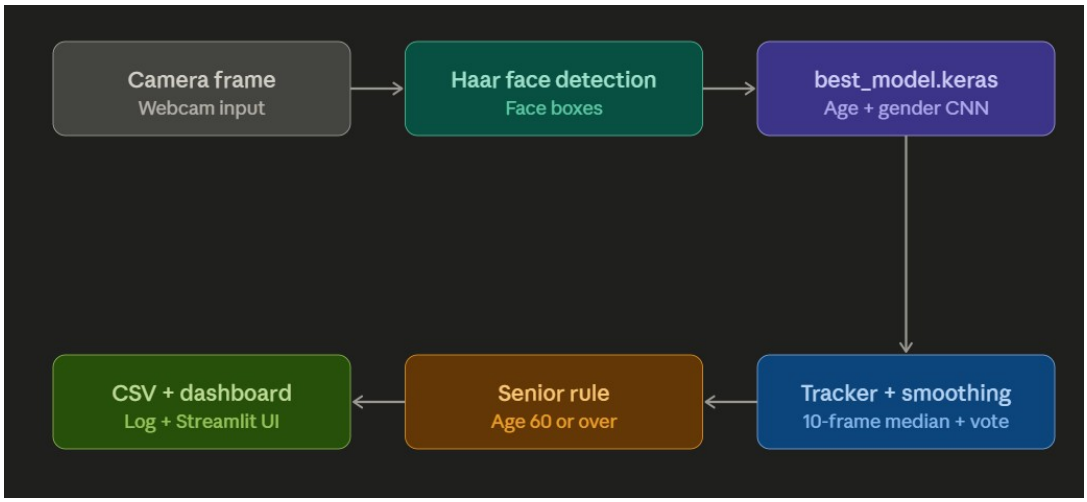


Figure 11 — Part 2 pipeline — camera frame → Haar face detection → best_model.keras (age + gender) → tracker with 10-frame smoothing → senior rule → CSV and Streamlit dashboard.

2.4 Reusing the Age + Gender Model

The single most useful decision here was not to train anything new. The model saved at the end of Part 1 already predicts gender and a continuous age from a face, so it is loaded exactly as saved and cached with Streamlit's @st.cache_resource so it is read from disk only once rather than on every frame. The one firm requirement is that each detected face is preprocessed in the identical way it was during training, otherwise the model would be fed inputs it never saw.

```
@st.cache_resource                          # load once, not on every frame
def load_model():
    return tf.keras.models.load_model("best_model.keras")   # reused from Part 1
model = load_model()

def predict(face_bgr):
    # identical preprocessing to Part 1: greyscale -> 128x128 -> /255
    gray = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, (128, 128))
    inp  = (gray / 255.0).reshape(1, 128, 128, 1)
    gender_pred, age_pred = model.predict(inp, verbose=0)
    gender = "Female" if float(gender_pred[0][0]) > 0.5 else "Male"
    age    = int(age_pred[0][0])
    return age, gender
```

Keeping the preprocessing byte-for-byte identical to Part 1 is what lets the existing model work on new, detector-cropped faces without any retraining. Gender comes back as a probability thresholded at 0.5, and age as a raw number rounded to a whole year.

2.5 Detecting Multiple People in a Frame

The reused model expects a single face, but a store camera shows a scene. A face detector bridges that gap by scanning each frame and returning a rectangle around every face it finds. OpenCV's Haar cascade detector was used because it ships with the library, needs no extra download, and runs fast enough for a live feed on an ordinary laptop. Its detectMultiScale call returns a list of boxes — one per face — which is exactly the multiple-person capability the task needs; the rest of the pipeline simply loops over that list.

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")

def detect_faces(frame):
    gray  = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
                                          minNeighbors=5, minSize=(60, 60))
    return [(x, y, x + w, y + h) for (x, y, w, h) in faces]   # one box per face
```

For crowded scenes or faces at an angle, a deep-learning detector such as OpenCV's res10 SSD model or MTCNN would catch more faces and is a natural upgrade. The Haar cascade was chosen for simplicity and zero setup, and because the detector only hands back boxes, it can be swapped without touching the model or the logging code.

2.6 The Senior-Citizen Decision Logic

The rule is deliberately small: a person is a senior citizen if their estimated age is 60 or above. It is applied to every detected face independently and — unlike Part 1, where hair length could override the true gender — it does not change the gender prediction at all. Gender here is always the model's real prediction. The threshold lives in one constant, so it is trivial to adjust.

```
SENIOR_THRESHOLD = 60                 # a single constant, easy to adjust

def is_senior(age):
    return age >= SENIOR_THRESHOLD    # 60 or older is flagged
```

2.7 Tracking and Temporal Smoothing

Two problems appear the moment the system runs on video rather than a single image: the same person reappears in dozens of consecutive frames, and a single frame's prediction can be noisy. Both are handled together. A lightweight tracker assigns each face a stable ID by matching detections between frames on how close their centres are, so one visitor keeps one identity.

```
def get_face_id(x1, y1, x2, y2, existing_ids, tolerance=60):
    """Match a detected box to a tracked face by centre distance;
       return a matched ID or create a new one."""
    cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
    for fid, data in existing_ids.items():
        fx, fy = data["center"]
        if abs(cx - fx) < tolerance and abs(cy - fy) < tolerance:
            existing_ids[fid]["center"] = (cx, cy)
            return fid
    new_id = len(existing_ids)
    existing_ids[new_id] = {"center": (cx, cy)}
```

```
    return new_id
```

Each tracked face then keeps a short rolling buffer of its last ten predictions. The displayed age is the median of those readings, which shrugs off the occasional wild frame, and the displayed gender is a majority vote. Until the buffer fills, the label simply reads “Analysing…”, and the visitor is written to the log only once — the moment the buffer is full and the result is trusted — not on every frame.

```
BUFFER_SIZE = 10                       # collect 10 frames before trusting a result
buf["ages"].append(age_raw)
buf["genders"].append(gender_raw)

stable_age    = int(np.median(buf["ages"]))                      # robust to
outliers
stable_gender = max(set(buf["genders"]), key=buf["genders"].count)  # majority vote

# log this visitor only once, after the buffer has filled
if len(buf["ages"]) == BUFFER_SIZE and not buf["logged"]:
    if is_senior(stable_age) or log_all:
        log_record(stable_age, stable_gender)
    buf["logged"] = True
```

This is the practical answer to the 60-year boundary worry: because the age output carries a few years of error, a single frame can flip a borderline person across the threshold, but the median over ten frames is far steadier and gives a much more reliable senior decision.

2.8 Logging Visits to CSV

The persistent output of the whole task is the file. On start-up the system creates senior_visit_log.csv with a header row if it does not already exist, and from then on each confirmed visitor appends a single line holding the time of visit, the estimated age, the gender, and whether they were flagged as a senior citizen.

```
CSV_FILE = "senior_visit_log.csv"

def log_record(age, gender):
    record = {
        "Timestamp":      datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "Age":            age,
        "Gender":         gender,
        "Senior Citizen": "Yes" if age >= SENIOR_THRESHOLD else "No",
    }
    pd.DataFrame([record]).to_csv(CSV_FILE, mode="a", header=False, index=False)
    return record
```

By default the log records seniors only, which keeps the file focused on the people the task cares about; a “log all persons” switch in the sidebar records every detected visitor instead. Each row is self-contained and timestamped, so the file doubles as a simple footfall record the store can open in Excel or analyse later. A few example rows:

Timestamp	Age	Gender	Senior Citizen
2026-06-23 14:02:11	67	Male	Yes

Timestamp	Age	Gender	Senior Citizen
2026-06-23 14:03:48	62	Female	Yes
2026-06-23 14:05:20	71	Male	Yes

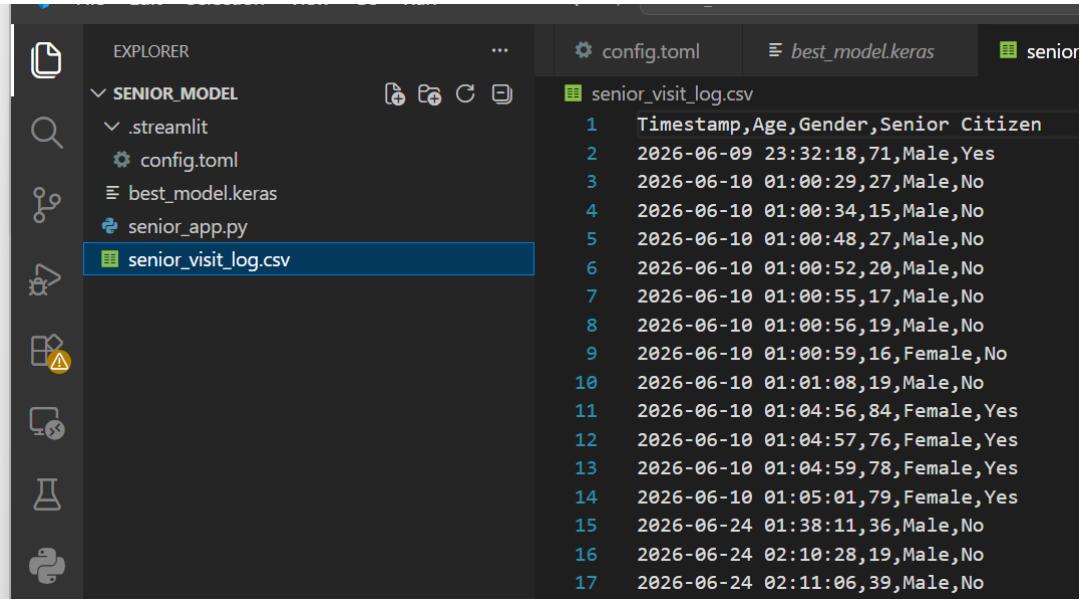


Figure 12 — The generated senior_visit_log.csv opened in an editor — real logged rows from a run, each with timestamp, age, gender and senior flag.

2.9 The Graphical Interface

Although the GUI is optional here, the system was given a full Streamlit dashboard so it can be run and demonstrated without touching the command line, styled with a dark theme so the red senior highlights stand out against the video. The interface offers two input modes selected with a single toggle.

Live webcam. A “Start Webcam” switch opens the camera and runs the detect → predict → smooth → log loop on the incoming frames, drawing boxes and labels straight onto the video. To keep it responsive, a “process every N frames” slider lets the user skip frames so only every Nth one is analysed.

```
run = st.toggle("Start Webcam")
if run:
    cap = cv2.VideoCapture(0)               # 0 = webcam
    frame_count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_count += 1
        if frame_count % frame_skip == 0:              # skip frames for speed
            boxes = detect_faces(frame)
            annotated, detects = annotate(frame.copy(), boxes, log_all)
            for age, gender in detects:
                log_record(age, gender)
            frame_box.image(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB))  # live view
    cap.release()
```

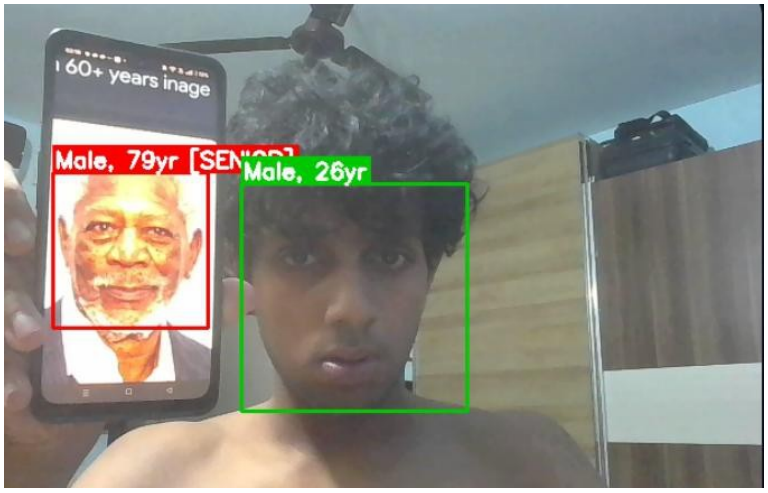


Figure 13 — Live webcam mode on a multi-person scene — senior citizens (60+) boxed in red, everyone else in green, each face labelled with gender and age.

Upload image. A single still photo is analysed in one shot — without temporal smoothing, since there is only one frame — and every detected face is boxed, labelled and listed in a small table. Alongside the video or image, a stats panel shows live session counts as coloured cards (total people logged, how many were seniors, the male/female split) with a table of the most recent detections underneath. A sidebar holds the controls — the "log all persons" switch, the frame-skip slider, a "clear log" button — plus a panel restating the model, detector, senior rule and smoothing settings. At the bottom, a full visit-log viewer reads senior_visit_log.csv back, shows running totals and the complete table, and offers a one-click CSV download, so the same file the detection loop writes is immediately viewable and exportable from the GUI.

Timestamp	Age	Gender	Senior Citizen
2026-06-10 01:00:48	27	Male	No
2026-06-10 01:00:52	20	Male	No
2026-06-10 01:00:55	17	Male	No
2026-06-10 01:00:56	19	Male	No
2026-06-10 01:00:59	16	Female	No
2026-06-10 01:01:08	19	Male	No
2026-06-10 01:04:56	84	Female	Yes
2026-06-10 01:04:57	76	Female	Yes
2026-06-10 01:04:59	78	Female	Yes
2026-06-10 01:05:01	79	Female	Yes

Figure 14 — The in-app visit-log viewer, showing the most recent detections with their timestamps, ages, genders and senior flags.

2.10 Challenges, Results and Conclusion

Reusing a model trained on clean crops means performance depends on conditions: it needs good lighting and a reasonably close, clear view of the face, and accuracy drops under poor lighting, extreme angles, or when the

face is far from the camera. Steadying noisy predictions at the 60 boundary was handled by the ten-frame median and majority vote, which made the live result noticeably more stable. Avoiding duplicate log rows required the proximity tracker plus a “log once per face appearance” flag, so video repetition does not fill the CSV with duplicates. Missed faces in crowds remain a limitation — Haar cascades miss profile and partially hidden faces — with a deep-learning detector the clear upgrade path that slots in without changing the model or the logging.

Part	Role	Result
best_model.keras (reused)	Age + gender per face	Same model as Part 1, no retraining required
OpenCV Haar detector	Find every face per frame	Enables true multi-person detection
Tracker + 10-frame buffer	One stable, de-duplicated entry per visitor	Median age + majority-vote gender, logged once
Senior rule + CSV logging	Flag age ≥ 60, write the file	senior_visit_log.csv with a timestamp per visitor
Streamlit dashboard	Webcam / upload UI, live stats, CSV download	Full optional GUI delivered

Part 2 reuses the Part 1 age-and-gender model unchanged and wraps it in the machinery a real deployment needs: face detection to handle whole scenes, tracking and smoothing to count each person once and steady the result, a one-line senior rule, durable logging, and a dashboard to drive and inspect it all. The carried-over lesson is that most of the work lives outside the model — in detecting, smoothing, deciding and recording — and that a clear pipeline is what turns a single prediction into a usable system. Future improvements would swap the Haar cascade for a deep-learning detector, extend the log with a unique visitor ID and a dwell time, and add a privacy notice, since a system that reads age and gender from people in a public space and writes it to a file carries real consent and data-retention obligations in a genuine deployment.

Part 3 — Nationality Detection System

Ethnicity classification, emotion recognition, age estimation and dress-colour detection. Custom Keras CNN · DeepFace · OpenCV · scikit-learn · Streamlit.

3.1 Problem Statement

The task is to build a nationality-detection system that accepts a face photograph, identifies the person's ethnicity (used here as a proxy for nationality, since nationality is not a visually inferable property), and then applies a set of conditional rules that decide which additional attributes to predict and display:

- **Indian** — predict emotion, age, and dress colour.
- **United States (White)** — predict emotion and age only.
- **African** — predict emotion and dress colour only.
- **Other nationality** — predict ethnicity and emotion only.

The system must also provide a proper graphical interface with an image preview and a dedicated output section showing all results for the predicted category.

3.2 Understanding the Problem

Reading the task carefully reveals that the word "nationality" is used loosely. Nationality is a legal concept tied to citizenship documents and cannot be inferred from a face image. What can be inferred, imperfectly, is a person's apparent ethnic or regional background. The system therefore classifies broad ethnicity categories and maps them to the four rules. This distinction is stated clearly because it matters for understanding both what the model can and cannot do and what the results actually mean. Once that reframing is in place, the task decomposes into four independent sub-problems:

- Ethnicity classification from a face image — to decide which rule to apply.
- Emotion recognition from a face image — required in every branch.
- Age estimation from a face image — required for the Indian and US branches.
- Dress-colour detection from the region below the face — required for the Indian and African branches.

No single model solves all four. The solution is a pipeline whose components run in sequence, with a short decision function implementing the branching logic. Identifying this structure early was the key design decision, turning a single ambiguous requirement into four clear, independently solvable sub-tasks.

3.3 System Overview

The system is a four-component pipeline wrapped inside a Streamlit application. When a user uploads a photograph, the face is located and cropped, the relevant models are run, and their outputs are merged by the decision layer before anything is shown. The table below summarises the components and, importantly, distinguishes what was self-built from what was integrated.

Component	How it works	Built or integrated
Ethnicity / nationality	Custom Keras CNN (race head) + skin-tone refinement	Self-built model
Emotion	DeepFace.analyze (pretrained)	Integrated library
Age	Keras CNN reused from Part 1	Self-built (Part 1)

Component	How it works	Built or integrated
Dress colour	OpenCV + scikit-learn KMeans + LAB naming	Self-built (classical CV)
Decision logic / GUI	Plain Python + Streamlit	Self-built

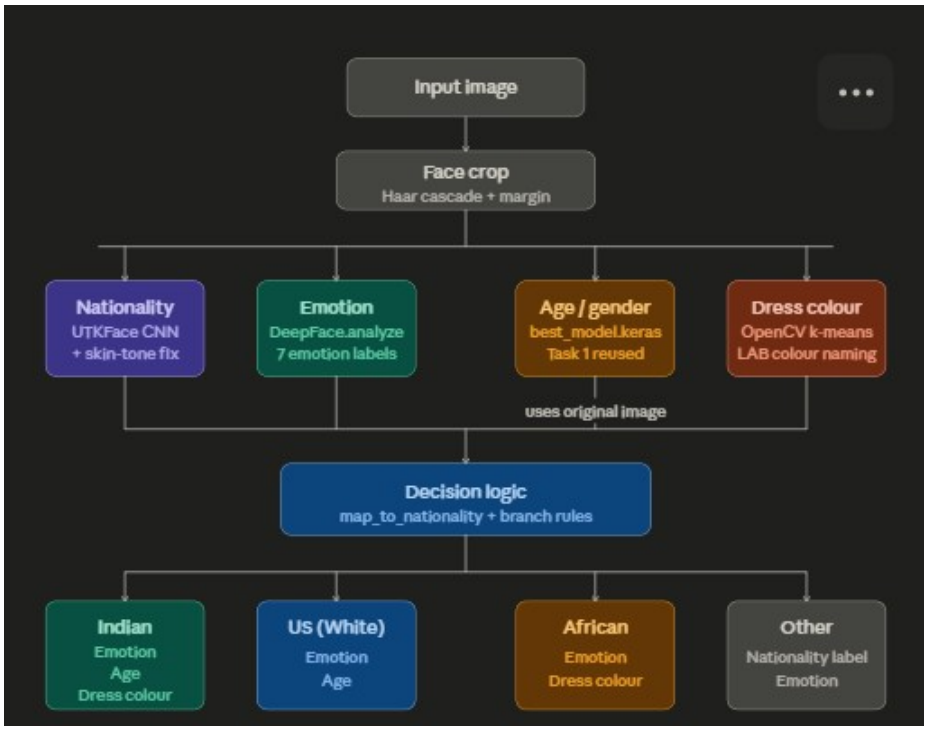


Figure 15 — The four-component pipeline — input → face crop → ethnicity, emotion, age and dress-colour components → decision logic → branch-specific output.

3.4 Datasets and Models Used

The ethnicity classifier and the age model are both built on UTKFace, in which age, gender and ethnicity are stored in the filename. UTKFace labels ethnicity in five classes — White, Black, Asian, Indian and Others — which is exactly the label set this project uses (a race_map maps indices 0–4 to those names). For the branch rules, Indian maps to the Indian branch, Black/African to the African branch, White to the US branch, and the remaining classes fall into Other. This mapping is a deliberate simplification recorded as a known limitation — a person of any apparent ethnicity may hold any national citizenship.

It is important to be accurate about the emotion model: none was trained for this project. Emotion recognition is handled entirely by DeepFace, an open-source facial-analysis library that ships with its own pretrained emotion classifier returning one of seven labels — happy, sad, angry, fear, disgust, surprise and neutral. The project integrates DeepFace rather than building an emotion network from scratch.

3.5 Component 1 — Ethnicity Classification

Ethnicity is predicted by a custom Keras convolutional model (loaded from age_gender_race_model.keras) that produces a race output over the five UTKFace classes. Both models are loaded once and cached at start-up.

```
@st.cache_resource
def get_models():
    age_gender_model = load_model('best_model.keras')           # Part 1
```

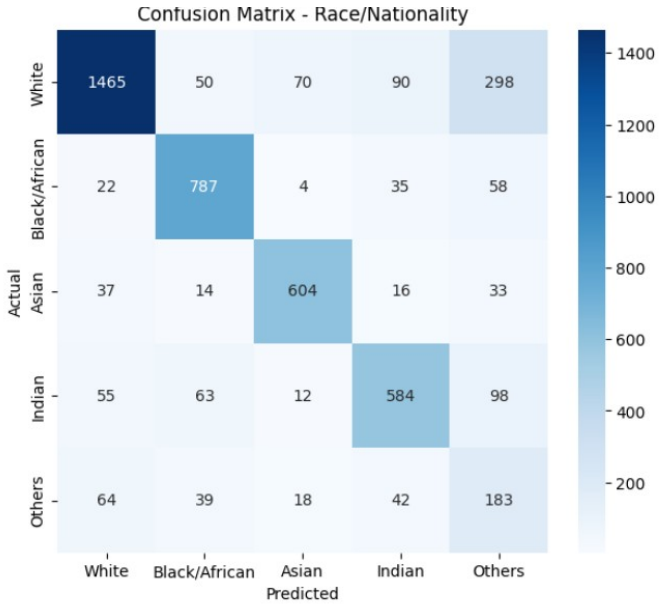
```
    nationality_model = load_model('age_gender_race_model.keras')
    return age_gender_model, nationality_model

race_map = {0: 'White', 1: 'Black/African', 2: 'Asian',
            3: 'Indian', 4: 'Others'}
```

Before the model runs, the face is cropped with a Haar cascade and normalised — white-balanced, converted to grayscale, resized to 128×128, and contrast-enhanced with CLAHE — which makes the classifier more robust to lighting differences. The raw race probabilities are then passed through a small refinement step. The African and Indian classes are the easiest to confuse, so when the model's top prediction is one of those two and the gap between them is small, the system measures the actual skin-tone brightness from the centre of the face and uses it to break the tie: darker toward African, lighter toward Indian. This is a deliberate hand-built correction on the model's single most common confusion.

```
def refine_with_skin_tone(probs, pil_face):
    top = int(np.argmax(probs))
    if top in (AFRICAN, INDIAN) and abs(probs[INDIAN] - probs[AFRICAN]) < 0.6:
        brightness = mean(get_skin_tone(pil_face))
        return AFRICAN if brightness < THRESHOLD else INDIAN
    return top
```

The refined class is mapped to a nationality branch (Indian → Indian, Black/African → African, White → US, everything else → Other), and the model's softmax probability for that class is shown as a confidence percentage. Evaluated on a held-out test set, the ethnicity model reaches roughly 76 percent overall accuracy. The per-class results are uneven: White, Black/African and Asian are recognised well (F1 around 0.81–0.85), Indian is moderate, and the catch-all "Others" class is weakest (F1 around 0.36), which is expected since it groups several visually different groups.



	precision	recall	f1-score	support
White	0.89	0.74	0.81	1973
Black/African	0.83	0.87	0.85	906
Asian	0.85	0.86	0.86	704
Indian	0.76	0.72	0.74	812
Others	0.27	0.53	0.36	346
accuracy			0.76	4741
macro avg	0.72	0.74	0.72	4741
weighted avg	0.81	0.76	0.78	4741

Figure 16 — Confusion matrix and classification report for the five-class ethnicity model. The "Others" class is the hardest, and the Indian / Black confusion is what the skin-tone refinement is designed to mitigate.

3.6 Component 2 — Emotion Recognition (DeepFace)

Emotion recognition is performed by DeepFace, a pretrained library, not by a model trained here. When a face image reaches the emotion function it is converted from RGB to BGR (the ordering OpenCV and DeepFace expect) and passed to DeepFace.analyze, which runs its own pretrained network and returns scores over seven emotion classes, of which the dominant one is taken.

```
def predict_emotion(pil_image):
    img = cv2.cvtColor(np.array(pil_image.convert('RGB')), cv2.COLOR_RGB2BGR)
    try:
        result = DeepFace.analyze(img, actions=['emotion'],
                                  enforce_detection=False)
        return result[0]['dominant_emotion'].lower()
    except Exception:
        return 'neutral'
```

Two choices are worth noting. The enforce_detection=False flag tells DeepFace to proceed even when it cannot cleanly find a face, which stops the app crashing on already-cropped or difficult images. And the whole call is wrapped in a try/except that falls back to 'neutral' on any error, so a failure in this one component never breaks the pipeline. To be clear: this component is integrated, not self-built — the classification is DeepFace's pretrained model; what is original here is the preprocessing, the safe error handling, and the presentation.

3.7 Component 3 — Age Estimation (reused from Part 1)

Age estimation reuses best_model.keras from Part 1 without retraining. In Part 3 the age is only computed when the predicted branch is Indian or US, exactly as the rules require. The preprocessing is kept identical to Part 1 — grayscale, resized to 128×128, scaled to 0–1 — because feeding the model anything else would give meaningless results. Because age from a face carries a few years of error, the interface displays the estimate with a "± 6 years" note rather than implying false precision.

```
age_input = preprocess(pil_image)          # grayscale 128x128, /255
ag = age_gender_model.predict(age_input)
gender = 'Male' if ag[0][0][0] < 0.5 else 'Female'
age = int(ag[1][0][0])
```

3.8 Component 4 — Dress-Colour Detection (Classical CV)

Dress colour does not need a neural network. The question — what is the dominant clothing colour — is answered directly by clustering the pixels below the face and naming the most prominent cluster, which is faster, needs no training data, and is easy to reason about. The clothing region is taken as the band of pixels below the detected face. That region is white-balanced, then filtered to remove very bright/near-white background pixels and skin pixels (via a YCrCb skin test), so the clustering focuses on actual clothing. The remaining pixels are clustered with KMeans, and each cluster is scored by how saturated it is and how many pixels it contains — so a small but vivid garment isn't lost to a large dull background.

```
km = KMeans(n_clusters=min(3, len(filtered)), n_init=10,
            random_state=42).fit(filtered)
hsv = rgb_to_hsv(km.cluster_centers_)
scores = hsv[:, 1] * np.sqrt(cluster_counts)      # saturation x size
dominant = centers[np.argmax(scores)]
```

The dominant cluster's RGB value is converted to a human-readable colour name by finding the nearest reference colour in CIELAB space, which matches human colour perception more closely than plain RGB distance. The reference palette covers seventeen common colours (black, white, gray, red, maroon, pink, orange, brown, tan, beige, yellow, olive, green, teal, blue, navy, purple).

```
def closest_color_name(rgb):
    px = rgb_to_lab(rgb)
    return min(COLOR_NAMES, key=lambda n: lab_distance(px, COLOR_NAMES[n]))
```

3.9 The Combined Decision Logic

The ethnicity classifier (after skin-tone refinement) decides the branch, and a short routing function then runs only the components that branch requires. Emotion is computed in every branch; age and dress colour are conditional.

```
result = {'nationality': nationality, 'emotion': predict_emotion(img), ...}

if nationality == 'Indian':
    result['age']   = age
    result['dress'] = detect_dress_color(img)
elif nationality == 'US':
    result['age']   = age
```

```
elif nationality == 'African':
    result['dress'] = detect_dress_color(img)
# 'Other' -> ethnicity label + emotion only
```

Emotion runs for everyone. The Indian branch adds both age and dress colour; the US branch adds only age; the African branch adds only dress colour; and the Other branch reports the ethnicity label alongside emotion. Each component does one job, and this short function is where the actual rule is satisfied — which matters because the brief weights correct logic as heavily as accuracy.

3.10 The Streamlit Interface

The interface is a two-column Streamlit layout: image upload and preview on the left, and a results panel on the right whose contents change with the predicted branch. The examples below show the app on real uploads, with the conditional cards (emotion, age, dress colour, ethnicity) appearing or not according to the detected nationality, alongside the race-model confidence bar.

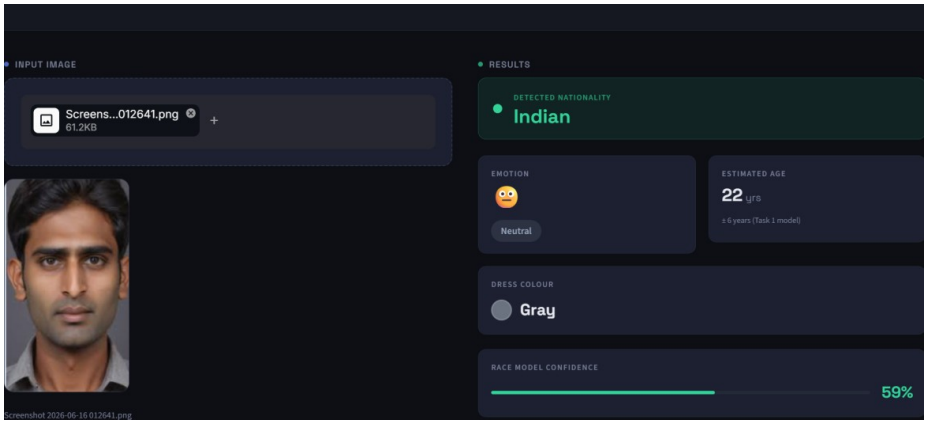


Figure 17 — Indian branch — emotion, estimated age and dress colour are all shown, with the race-model confidence below.

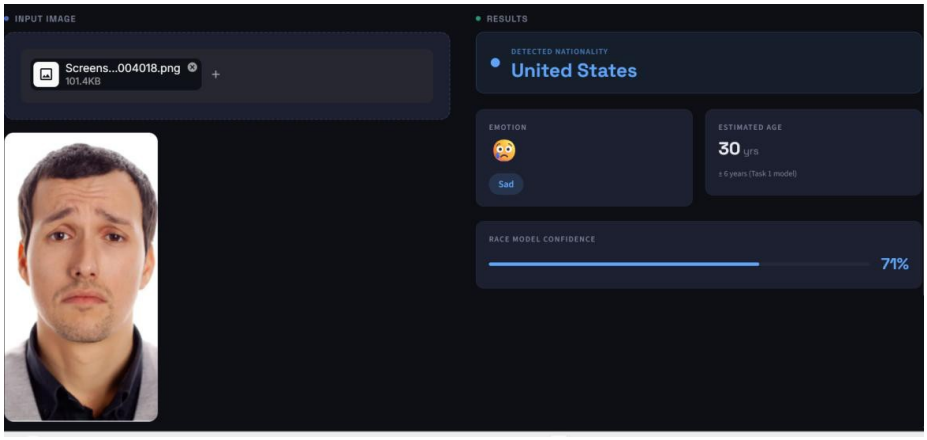


Figure 18 — United States branch — only emotion and age are shown, as the rule specifies.

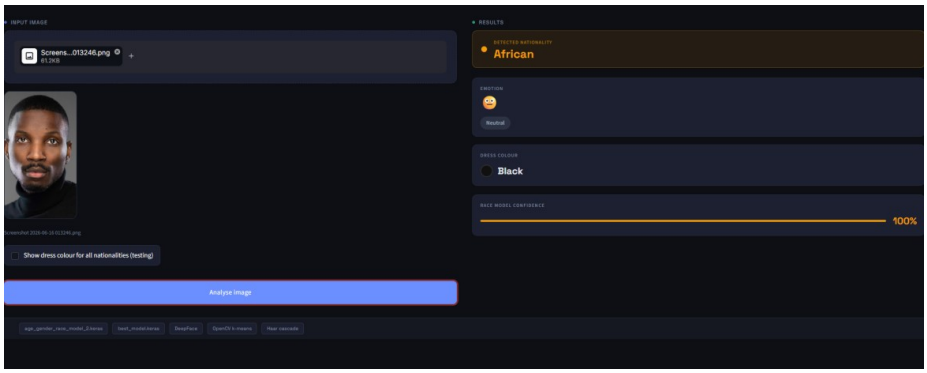


Figure 19 — African branch — emotion and dress colour are shown; age is omitted.

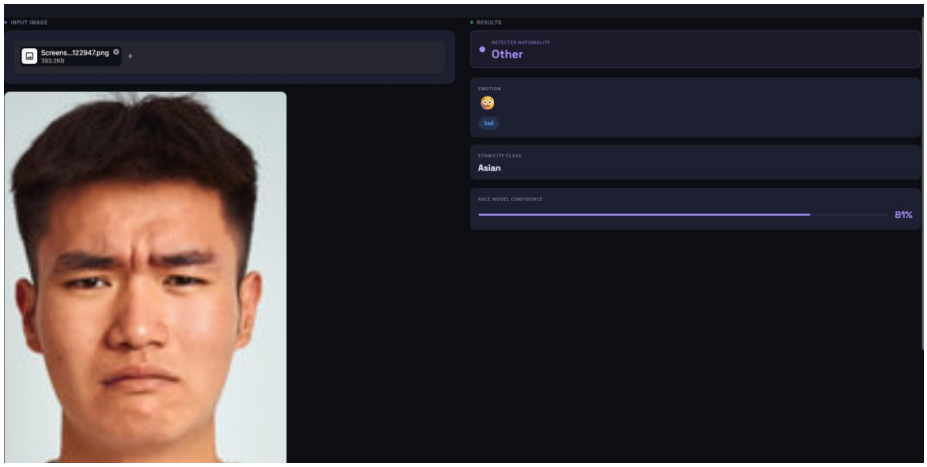


Figure 20 — Other branch — the ethnicity label and emotion are reported, with no age or dress colour.

3.11 Challenges, Results and Conclusion

The recurring honesty about nationality versus ethnicity is itself a challenge handled deliberately: nationality has no visual signal, so the system classifies apparent ethnicity and states that limitation plainly rather than pretending to read citizenship. The Indian-versus-African confusion — the model's most common mix-up — was addressed with a targeted skin-tone brightness tie-breaker that only activates when the model is genuinely unsure between those two classes, a patch on a known weakness rather than a replacement for the model. Isolating clothing from skin and background required a YCrCb skin filter and dropping near-white background pixels, then scoring clusters by saturation as well as size. Running self-built Keras models alongside the DeepFace library in one process was kept responsive by caching every model at start-up and keeping each component's preprocessing separate.

Component	Role	Result
Ethnicity (mine, Keras)	Routes to the correct branch	~76% test accuracy (5 classes); strong on White/Black/Asian, weak on "Others"
Emotion (DeepFace)	Emotion in every branch	Integrated pretrained model (7 emotions)
Age (mine, Part 1)	Age for Indian/US branches	Reused Part 1 CNN, ~6 yr error
Dress colour (classical CV)	Colour for Indian/African	KMeans + LAB naming, 17 colours
Whole system	Branch → conditional attributes	Correct routing, shown in the GUI

Part 3 shows that a multi-branch prediction system can be built cleanly by decomposing the requirement into independent components and combining them with a small, readable decision function — the branching logic is trivial once each component is in place, and the real work is choosing the right tool for each sub-problem and being honest about which parts are trained and which are integrated. The ethnicity classifier and the age model are custom Keras networks (the age model reused from Part 1), the dress-colour detector is a custom classical-CV routine, and the emotion component is DeepFace's pretrained model behind a safe wrapper. Future improvements would train a dedicated emotion model so the whole pipeline is self-built, strengthen the weak "Others" ethnicity class with more data, replace the Haar cascade with a deep face detector, and adapt the dress-colour region to where the body actually is.

Part 4 — Age and Emotion Detection through Voice

From a self-built classical pipeline to a deep-learning upgrade. librosa · scikit-learn · wav2vec2 (PyTorch) · Streamlit.

4.1 The Task

The brief was to detect a person's age from a voice note, with a few rules wrapped around it. The system only handles male voices — a female voice must be rejected with "Upload male voice." If the speaker is over 60, they are marked as a senior citizen and their emotion is also worked out. If they are under 60, the system just reports the age and stops. As with the vision tasks, the brief stresses that this is mainly a test of logic-building and problem-solving, that the model must be self-built, and that evaluation is based on how the whole thing behaves rather than a single accuracy number — which changes what "done" means: a clean, well-reasoned pipeline matters more than chasing one big percentage.

4.2 Breaking It Down

This is not one problem but three, stacked, and each one only matters depending on what the one before it said. To answer the brief for any clip, three things are needed in order: gender first, because a female voice is bounced straight away; then age, because that decides senior-or-not and whether emotion even runs; then emotion, but only if the voice is male and 60 or over. So no single model does this — it is a pipeline of three models with routing logic between them.

```
if gender == 'female':
    say 'Upload male voice.'          # stop here
else:                                 # male
    age = predict_age(voice)
    if age is senior (60+):
        emotion = predict_emotion(voice)
        show senior + emotion
    else:
        show age group
```

An early decision was not to predict an exact age in years from the classical model. Even serious research systems are off by several years on voice-age, so an exact number from hand-built features was never realistic. Instead the classical model predicts age groups, which the dataset already provides and which is all the senior rule actually needs.

4.3 Phase 1 — The Classical Pipeline

For the gender and age models I started with Mozilla Common Voice, a large, free, publicly available speech dataset — the obvious first choice, since it is openly downloadable and ships with age and gender labels. It did not take long to run into its limits. Common Voice is crowd-sourced audio of people simply reading sentences aloud, recorded on whatever laptop or phone microphone each contributor happened to have, so the recording quality is inconsistent from clip to clip and there is no emotional content in it at all. The models I trained on it looked fine on the dataset's own clips but turned out very inaccurate the moment they met anything else — which is what eventually sent me searching for a better option and, ultimately, reshaped the whole approach (Sections 4.4–4.5).

The first version (app.py) is built entirely from classical machine learning: acoustic features are pulled from each clip by hand with librosa, then Random Forest classifiers are trained on them. This was a deliberate choice — it

is light, trains in minutes on a normal CPU, exposes which features matter, and forces genuine reasoning about what separates the classes, which is the skill the brief was testing. All three Phase 1 models share this approach.

Gender model

Gender was the easy one, because pitch does most of the work — men's voices sit much lower than women's. For each clip the pipeline extracts 13 MFCCs (a compact summary of the voice's tone), the average pitch using librosa's pyin, and a few spectral measures; a Random Forest learns the rest.

```
def extract_features(file_path):
    y, sr = librosa.load(file_path, sr=16000)
    mfcc  = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
    f0, _, _ = librosa.pyin(y, fmin=..., fmax=...)     # pitch
    # + spectral centroid, bandwidth, zero-crossing rate
    return feature_vector
```

This landed around 89–90 percent on held-out data. It did better on male voices than female ones, which comes straight from the data — Common Voice has far more labelled male clips. Class weighting and a richer feature set with jitter and shimmer plus audio augmentation were tried, but the augmented version did worse on real voices, so it was dropped in favour of the simpler model — a deliberate, measured call rather than laziness.

Age model

Age was where most of the effort went. Training used male clips only, since that is the only path that reaches the age stage. The four working-age decades were kept as their own classes (twenties, thirties, forties, fifties) and everything 60-and-up collapsed into a single senior class, because the senior flag is all the brief needs from the older end, and merging the tiny, data-starved older buckets into one bigger class helped.

```
SENIOR_SOURCE = {'sixties', 'seventies', 'eighties', 'nineties'}
KEEP_BUCKETS  = {'twenties', 'thirties', 'fourties', 'fifties'}
# 60+ -> 'senior_60plus', keep the rest as-is, drop teens
```

The big problem was imbalance — the twenties and thirties buckets dwarf everything else. A first attempt on a plain random sample scored about 53 percent and was hopeless on the rare classes. The fix that moved the needle was per-class sampling: take every example available from the rare buckets, and cap the common ones. All real data, no synthetic anything.

```
def build_balanced_sample(df, max_per_class=1500):
    parts = []
    for label, grp in df.groupby('age_label'):
        parts.append(grp.sample(min(len(grp), max_per_class)))
    return pd.concat(parts)
```

That pushed it to about 63 percent, confirmed with 5-fold cross-validation that barely moved between folds, so it is a stable number rather than a fluke. The senior class specifically hit around 80 percent precision — when it says senior, it is usually right, which is the property that matters for gating emotion. Its recall was lower, meaning it misses some real seniors and calls them younger — a real weakness, but a safe one, since it under-flags rather than over-flags.

```
Accuracy: 0.6453333333333333
              precision    recall  f1-score   support

      fifties       0.71      0.77      0.74       300
     fourties       0.63      0.69      0.66       300
senior_60plus       0.71      0.81      0.76       300
     thirties       0.56      0.51      0.53       300
     twenties       0.57      0.45      0.50       300

     accuracy                           0.65      1500
    macro avg       0.64      0.65      0.64      1500
 weighted avg       0.64      0.65      0.64      1500

[[232  20  13  15  20]
 [ 19 207  16  35  23]
 [ 12  10 243  17  18]
 [ 33  41  33 152  41]
 [ 31  48  36  51 134]]
Classes: ['fifties' 'fourties' 'senior_60plus' 'thirties' 'twenties']

CV accuracy: 0.633 ± 0.008
```

Figure 21 — Phase 1 age model — classification report and confusion matrix. Overall ≈ 63% (5-fold CV ≈ 0.633 ± 0.008); the senior class shows high precision.

Emotion model

Common Voice has no emotion labels — people just read sentences — so emotion used CREMA-D, where actors perform emotions and the label is baked into the filename. CREMA-D was chosen over easier options like TESS on purpose: TESS has only two speakers and gives a fake-looking 99 percent, while CREMA-D has 91 speakers of both genders and a wide age range, much closer to the senior-male voices this stage actually sees. Because emotion lives in how a voice moves rather than just its average, a richer feature set was used here — 40 MFCCs with both mean and standard deviation, deltas, chroma and mel features, plus energy and pitch stats: 163 features in total. The emotions were also scoped down. A four-class version including neutral got stuck around 71 percent because neutral is muddy and bleeds into everything; dropping neutral and keeping the three clear ones — angry, happy, sad — pushed it to about 77 percent. This is an upfront three-class result, scoped on purpose. Augmentation was tried here too, the proper way (only on the training split), and again made things slightly worse, so it was reverted — the same lesson as the gender model.

```
Accuracy: 0.7745740498034076
              precision    recall  f1-score   support

       angry       0.77      0.79      0.78       255
       happy       0.68      0.65      0.67       254
         sad       0.87      0.88      0.87       254

    accuracy                           0.77       763
   macro avg       0.77      0.77      0.77       763
weighted avg       0.77      0.77      0.77       763

[[202  50   3]
 [ 57 166  31]
 [  2  29 223]]
Saved emotion model + scaler
```

Figure 22 — Phase 1 emotion model on CREMA-D (angry / happy / sad) — classification report and confusion matrix, ≈ 77% accuracy.

How well Phase 1 worked

On the dataset's own clips, gender was good, emotion decent, and age the weak link. But the moment it was tested on real voices — an old man's voice played off YouTube through a phone speaker, or a self-recording — it fell apart, calling clearly elderly men female and putting a 21-year-old voice in the 40s. That is not a coding bug but two real problems: phone-speaker audio loses the low frequencies that mark a voice as male, and the models only ever saw clean dataset clips, so anything else is foreign to them.

4.4 Why Phase 1 Wasn't Good Enough

The ceiling on the classical version is mostly baked into the problem, not a mistake in the build. First, age-from-voice is just hard: gender has a clean signal in pitch (which is why it hits 90 percent), but age does not — two people of very different ages can sound similar, and neighbouring decades overlap even to a human ear. Research using far heavier deep-learning methods still lands at roughly 7 to 11 years of average error, so 63 percent grouped accuracy is about what this kind of approach can do. Second, the data is thin where it hurts: the labelled part of Common Voice is mostly young adults, and elderly male voices — exactly what the senior flag depends on — are among the rarest; the free datasets with reliable age labels for older speakers essentially do not exist (the good ones, TIMIT and NIST, are behind licences). Third, real-world recordings wreck it: a phone speaker, a laptop mic or a compressed clip all sound different enough that the models misread them, especially at the gender gate. This is the standard problem with any model trained on one source, and it is the main reason the live demo was unreliable even though the test numbers were fine.

My first instinct was that a better dataset would fix all this, so I spent a good while hunting for one. But that search kept dead-ending: the good age-labelled corpora are behind licences, and swapping one clean read-speech collection for another would still leave me with clean read speech that generalises badly to real voices. After a lot of reading, the conclusion kept coming back the same way — the dataset was not the whole problem, the method was. To get genuinely accurate age and emotion from a voice, I needed a deep-learning model pretrained on a huge amount of varied speech, one that learns its own features straight from the raw waveform instead of the hand-crafted features I was feeding a Random Forest. That realisation, reinforced by a nudge from my mentor, is what took the project to Phase 2.

4.5 Phase 2 — The Deep-Learning Version

A classical model relies on features designed by hand; a deep model learns its own features straight from the raw audio waveform, because it has been pre-trained on a huge amount of speech first. wav2vec2 is one of these — a transformer that takes raw audio and produces an internal representation, which then feeds small prediction heads for age, gender or emotion. This project uses pretrained wav2vec2 models published by audEERING — one for age/gender, one for emotion — and integrates rather than trains them. Loading and running a state-of-the-art transformer, getting its inputs and outputs right, and then critically comparing it against the self-built work is real engineering; the claim is not that the transformer was built. The classical pipeline remains the core deliverable, with the deep model an upgrade bolted on and benchmarked. The weights are pulled from Hugging Face by name at runtime, so nothing is downloaded into the project folder in advance.

```
AGE_MODEL = 'audeering/wav2vec2-large-robust-6-ft-age-gender'
EMO_MODEL = 'audeering/wav2vec2-large-robust-12-ft-emotion-msp-dim'
# loaded with transformers + torch, run on CPU or GPU
```

The self-built gender model was kept as the gate, because it works well and the deep age model's gender output looked unreliable on the test clips. So the hybrid in app2.py is: the classical Random Forest decides male/female, then if it is male the wav2vec2 model estimates the age in years, and if that age clears the senior line the wav2vec2 emotion model runs. One honest quirk: the deep age model tends to underestimate older voices (it called several real seniors mid-40s), so the senior threshold was dropped from 60 to 52 in the app — a deliberate calibration to catch more genuine seniors at the cost of a few false positives. The emotion model also does not output words; it outputs three numbers — arousal, dominance and valence, the dimensional model of emotion used in research — which a small mapping turns into a plain label for the interface, with both the label and the raw scores shown.

Benchmarking against the self-built model

This is the most worthwhile part. Scoring the deep age model the same way as the classical one — bucket accuracy — gave 38 percent, which looked terrible. But that is the wrong ruler: the deep model predicts a continuous age, and hard decade boundaries punish it brutally (a true fifties person predicted as 48 counts as a complete miss even though it is two years off). Measured properly, with mean absolute error — the metric the research actually uses — it came out at 10.6 years, right inside the 7–11 year range published wav2vec2 results report. So the deep model is genuinely good at general age estimation and clearly stronger than the classical one in that sense. The honest twist is that for the specific job — catching seniors — even this model struggled, because it systematically underestimates elderly voices. Two completely different model types, a Random Forest and a deep transformer, both hit the same wall on old voices, which points to a fundamental limit of reading age from voice for older speakers rather than a modelling failure.

Age model	How it was measured	Result
Phase 1 — Random Forest	Bucket accuracy (5 classes)	~63% (stable across 5-fold CV)
Phase 2 — wav2vec2 (audEERING)	Mean absolute error (years)	10.6 years (in line with research)
Both	Catching real seniors (60+)	Weak — elderly voices underestimated

4.6 The Two Apps, Side by Side

The project ends with two working applications, kept on purpose because they show different things.

	app.py (Phase 1)	app2.py (Phase 2)
Gender	Random Forest	Random Forest (same model)
Age	Random Forest (buckets)	wav2vec2 (age in years)
Emotion	Random Forest (3-class)	wav2vec2 (dimensional)
Needs	scikit-learn + librosa	also torch + transformers
Speed	Instant	Slower (deep models, esp. on CPU)
What it proves	Building a pipeline from scratch	Integrating and benchmarking SOTA

app.py is fully self-built and runs anywhere with no heavy dependencies; app2.py is the hybrid that swaps in the deep models for age and emotion. Together they make the point better than either alone: the system was built from scratch, and there is also a clear sense of when and how to reach for a stronger pre-trained model.

4.7 The Interface

The GUI is built in Streamlit, with two input modes — record straight from the mic, or upload a file — running the clip through the pipeline and showing each stage's result. Female voices get the rejection message; male voices show an age, and seniors additionally show emotion. For app2.py the interface was redesigned into a full-width "neural voice analysis" console: a two-column layout with the input and a numbered pipeline rail on the left, and a live results panel on the right that fills in stage by stage. The result cards show the gender with a confidence bar, the age as a large readout with a senior badge, and the emotion as a colour-coded label plus bars for the three dimensional scores.

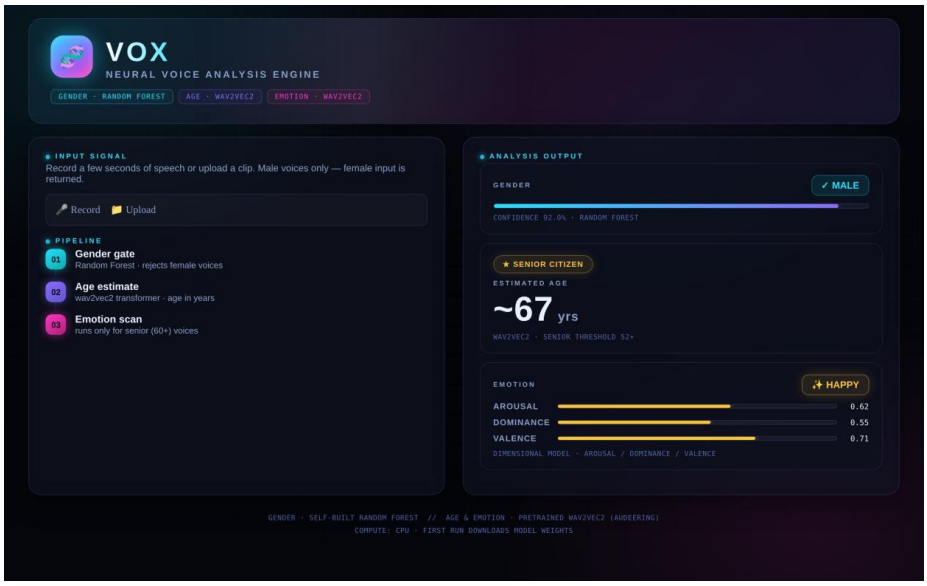


Figure 23 — Phase 2 console — a male senior voice (~67 yrs) crossing the threshold, with the emotion stage returning 'happy'.

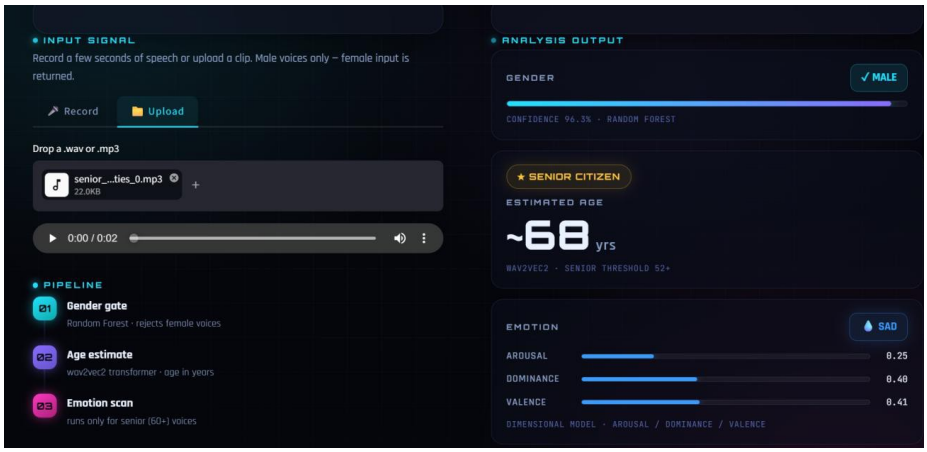


Figure 24 — A second senior result (~68 yrs) with the emotion stage returning 'sad', and the arousal / dominance / valence bars shown.

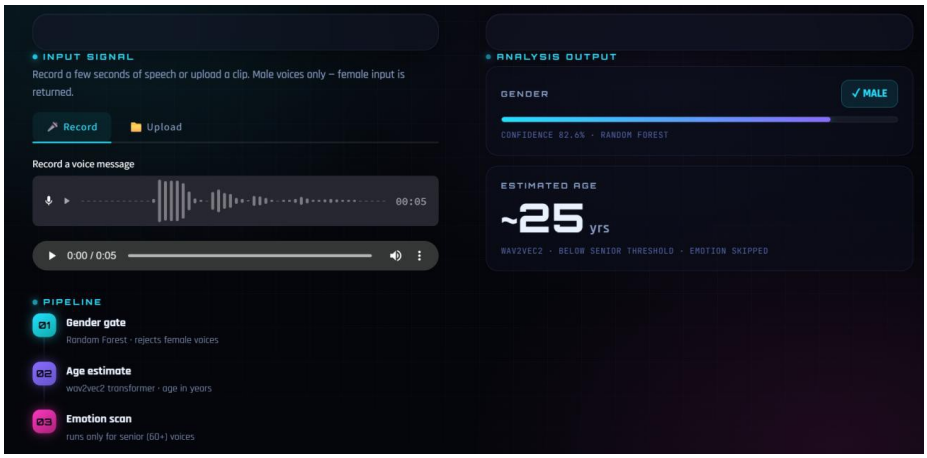


Figure 25 — A male voice below the senior threshold (~25 yrs) — the age is reported and the emotion stage is skipped.

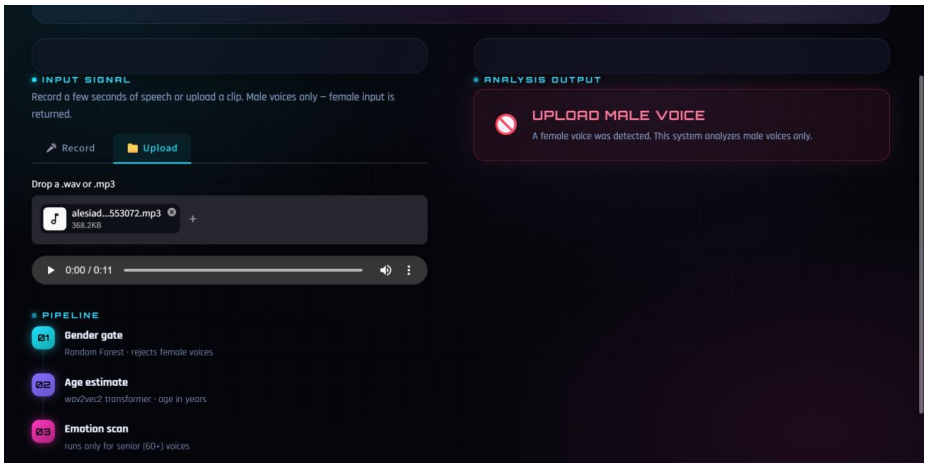


Figure 26 — The gender gate in action — a female voice is detected and rejected with "Upload male voice".

One implementation detail matters: the gender/age path and the emotion path use different feature pipelines (the emotion model needs a much richer input), so they are kept strictly separate. Feeding the wrong feature vector to a model does not error — it silently produces nonsense — so this separation is deliberate.

4.8 Challenges, Results and Conclusion

Class imbalance was the biggest challenge — Common Voice is mostly young adults and mostly male, so the older buckets are tiny, and per-class sampling across every split was how the most was made of what existed. A multi-hour feature-extraction run was lost early when the session disconnected before saving, after which features were cached to CSV and the loop checkpointed every few hundred rows, so an interruption costs almost nothing. Pitch-shift augmentation was tried twice, for gender and emotion, and both times made real-world performance worse, so both were reverted — the lesson being that more engineering is not automatically better and every change has to be measured. Getting torch and transformers to run locally on a Windows/Anaconda setup was genuinely painful, and was fixed by building a clean conda environment with pinned, compatible versions away from the tangled base environment.

Part	What it does	Result
Gender (mine)	Male / female gate	~89–90% test accuracy
Age (mine, Phase 1)	Age buckets + senior flag	~63%, stable on 5-fold CV
Emotion (mine, Phase 1)	Angry / happy / sad	~77% on CREMA-D (3 classes)
Age (wav2vec2, Phase 2)	Age in years	10.6 yr MAE (research-grade)
Whole system	Gate → age → senior → emotion	Correct routing, shown in both GUIs

This started as one confusing sentence and turned into a small system. The real insight was seeing it as three gated stages instead of one model, and much of the work was in wiring them together and then being honest about where each one breaks. Phase 1 is a from-scratch pipeline — gender, age and emotion on hand-made features and Random Forests, with a working GUI — that does what the brief asks. Phase 2 is the upgrade: pretrained wav2vec2 models swapped in for age and emotion, a research-grade 10.6-year age error, and an honest benchmark against the self-built model. The same lesson was learned twice (augmentation can quietly hurt), and the elderly-voice problem proved a real ceiling that even a deep transformer does not escape. Clear next steps are a bigger, age-balanced dataset of older speakers, possibly fine-tuning wav2vec2 rather than using it as-is, and noise-robust preprocessing so the live demo holds up better on phone and web audio.

Part 5 — Car Colour Detection & Traffic Analysis System

A multi-model approach combining object detection, deep-learning colour classification and pedestrian detection. Built with OpenCV, TensorFlow and Streamlit.

5.1 Problem Statement

The task is to build a traffic-analysis system that detects cars and pedestrians in images taken at traffic signals, following specific visual rules for how each detected object is marked:

- Detect all cars visible in an uploaded traffic image.
- Classify each car's colour — specifically identify whether it is blue or any other colour.
- Draw a red bounding box around blue cars and a blue bounding box around all other cars.
- Detect and count any people (pedestrians) present at the traffic signal.
- Display the total number of cars, blue cars, other cars, and people in a summary panel.
- Provide a graphical interface for uploading images and viewing results.

The task explicitly excludes YOLO-based detectors, and the evaluation places weight on the overall behaviour of the pipeline and the interface, making it as much a system-design exercise as a modelling one.

5.2 Understanding the Problem

On the surface this looks like a single detection problem, but it is really three separate problems stacked together. To produce the required output the system must locate every car and draw a box around it (object detection), determine the colour of each detected car by classifying the cropped region inside its box (image classification), and locate every person in the same image (pedestrian detection, which is separate from car detection and needs different trained features). No single model solves the task: one model detects cars and people, a second classifies the colour of each car crop, and a short piece of logic combines those results into the final annotated image and summary counts.

5.3 System Overview

The system is made up of two detection models, one classification model, and one decision layer, all wrapped inside a single Streamlit interface. When a user uploads a photo, it is passed through both detection models; each detected car crop is sent through the colour classifier; and the results are combined before anything is shown.

Component	Model / Technique	What it produces
Car and bus detection	MobileNet-SSD (CNN, Caffe)	Bounding boxes for every car/bus in the image
Colour classification	MobileNetV2 (CNN, TensorFlow — trained on VCoR)	Blue or other label for each detected car crop
Pedestrian detection	HOG + SVM (classical ML) + MobileNet-SSD person class	Bounding boxes and count for people
Decision and annotation	Plain Python (OpenCV drawing)	Red/blue/green boxes drawn on the image
GUI	Streamlit	Image upload, result display, metric summary

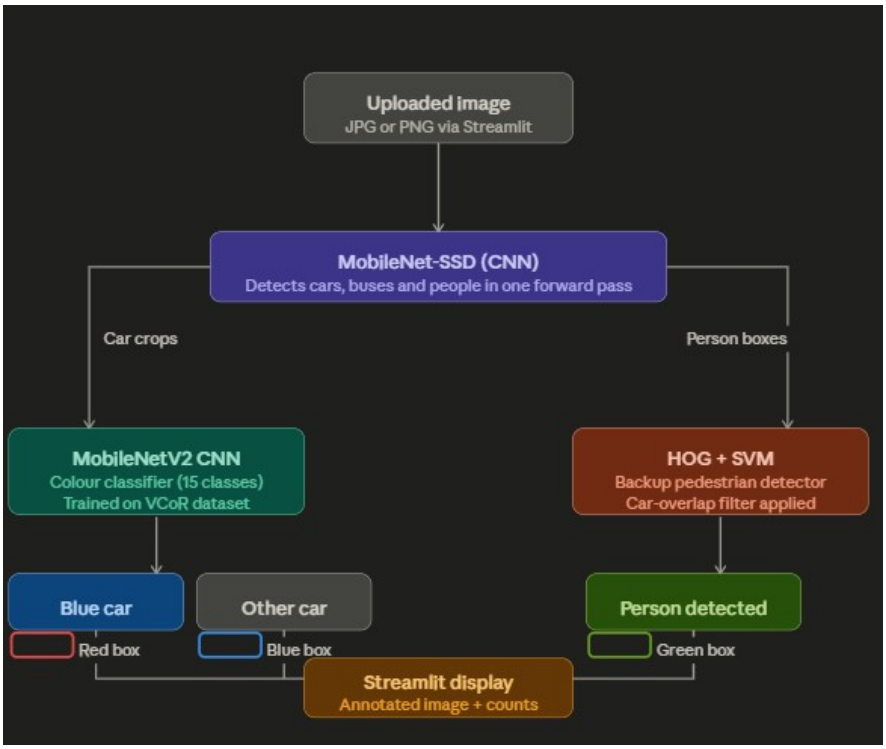


Figure 27 — System pipeline — the uploaded image flows through MobileNet-SSD detection, then car crops go to the MobileNetV2 colour classifier and person boxes to the HOG + SVM backup, before annotation and the Streamlit display.

5.4 Why YOLO Was Not Used

YOLO is a family of real-time detectors widely used in traffic applications and a natural first choice, but the project guidelines explicitly prohibit it. MobileNet-SSD (a Single Shot MultiBox Detector with a MobileNet backbone) was selected as the car and person detector instead, for several reasons: it is a genuine pretrained CNN rather than a rule-based or classical approach, so it fully satisfies the machine-learning requirement; like YOLO it is a single-stage detector performing detection in one forward pass; its pretrained Caffe model, trained on PASCAL VOC 2007 and 2012, already includes 'car', 'bus' and 'person' as classes with no retraining; and it can be loaded and run entirely through OpenCV's built-in DNN module, without a separate deep-learning framework. Initial testing showed 0.95–1.00 confidence on clear eye-level street photographs. MobileNet-SSD is architecturally different from YOLO — different backbone, anchor design and loss — so it is an independent, well-established detector, not a YOLO variant by another name.

5.5 Car Detection — MobileNet-SSD

The natural question at the start was whether an existing dataset could be used to train a detector from scratch. Several were investigated — UA-DETRAC, the KITTI benchmark, and MS COCO — but none were viable: UA-DETRAC and KITTI require tens of gigabytes, dedicated annotation tools and custom loaders, and are not cleanly available on Kaggle; MS COCO is on Kaggle but training a detector from scratch on it needs multi-day GPU runs; and smaller sets such as Stanford Cars and BoxCars annotate vehicle type rather than providing detection boxes. Using a pretrained MobileNet-SSD is standard, well-accepted practice: when a high-quality pretrained detector already exists for the required classes, reusing it and focusing effort on the problem-specific parts produces a better overall system than reproducing detection from scratch.

MobileNet-SSD uses a depthwise-separable convolutional backbone to extract feature maps at multiple scales, followed by multi-scale default anchor boxes at different aspect ratios; a single forward pass predicts class

probabilities and box offsets for all anchor positions, and Non-Maximum Suppression then filters overlapping detections, keeping the highest-confidence box per object. Input images are resized to 300×300, normalised to [-1, 1], and passed through the network, which outputs 21-class probabilities (background plus 20 PASCAL VOC classes). Detections above a user-set threshold are drawn onto the output image.

```
def run_ssd(image_bgr, x_off=0, y_off=0):
    h_img, w_img = image_bgr.shape[:2]
    blob = cv2.dnn.blobFromImage(
        cv2.resize(image_bgr, (300, 300)),
        scalefactor=0.007843, size=(300, 300),
        mean=(127.5, 127.5, 127.5), swapRB=False,
    )
    net.setInput(blob)
    detections = net.forward()
    # parse class_id, confidence, and bounding box per detection
    # return list of (class_id, confidence, x1, y1, x2, y2)
```

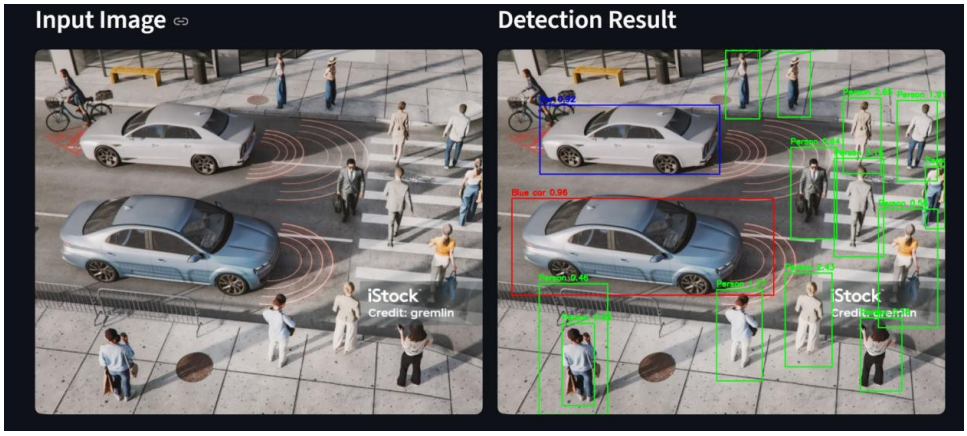


Figure 28 — Car detection output — colour-coded bounding boxes on a test traffic image (input on the left, detection result on the right).

5.6 Car Colour Classification — MobileNetV2

A separate trained model classifies the colour of each detected car, using the VCoR (Vehicle Color Recognition) dataset by Landry Kezebou, available on Kaggle. It contains thousands of car images pre-organised into subfolders by colour class, with train/validation/test splits already provided, so it loads straight into Keras's image_dataset_from_directory with no custom annotation.

Property	Detail
Source	Kaggle — landrykezebou/vcor-vehicle-color-recognition-dataset
Number of classes	15: beige, black, blue, brown, gold, green, grey, orange, pink, purple, red, silver, tan, white, yellow
Input image size	128 × 128 px (resized during loading)
Dataset splits	train / val / test (pre-split by dataset author)
Training platform	Kaggle Notebook with GPU acceleration

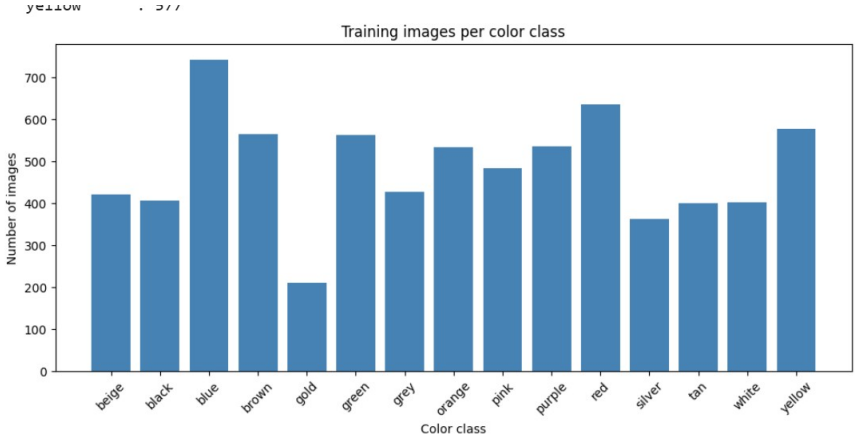


Figure 29 — Class distribution of the VCoR training set across all 15 colour classes.

Figure 30 — A representative sample image from each of the 15 VCoR colour classes.

Rather than training from scratch, MobileNetV2 pretrained on ImageNet was used as a feature extractor — a standard transfer-learning approach where the lower-level feature detectors from ImageNet (edges, textures, shapes, gradients) are reused and only the top classification layers are trained on the VCoR data. This suits car colour well, since it depends heavily on mid-level texture and colour features that transfer from ImageNet. The full architecture is a 128×128×3 input, data augmentation (random horizontal flip, 5% rotation, 10% zoom, 10% contrast), rescaling to [-1, 1], the MobileNetV2 backbone (frozen in Phase 1, top 30 layers unfrozen in Phase 2), Global Average Pooling, Dropout (0.3), and a 15-unit softmax output layer.

Training ran in two phases. Phase 1 froze the backbone and trained only the classification head for 10 epochs with Adam at 1e-3, quickly converging the new head without disrupting the pretrained extractor. Phase 2 unfroze the top 30 layers and fine-tuned them for 5 epochs at a much lower 1e-5, letting the higher-level feature maps adapt to car paint colours and lighting while avoiding catastrophic forgetting.

```
# Phase 1 - head only
base_model.trainable = False
model.compile(optimizer=Adam(1e-3), loss='categorical_crossentropy',
              metrics=['accuracy'])
history1 = model.fit(train_ds_p, validation_data=val_ds_p, epochs=10)
```

```
# Phase 2 - fine-tune top 30 layers
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=Adam(1e-5), loss='categorical_crossentropy',
              metrics=['accuracy'])
history2 = model.fit(train_ds_p, validation_data=val_ds_p, epochs=5)
```

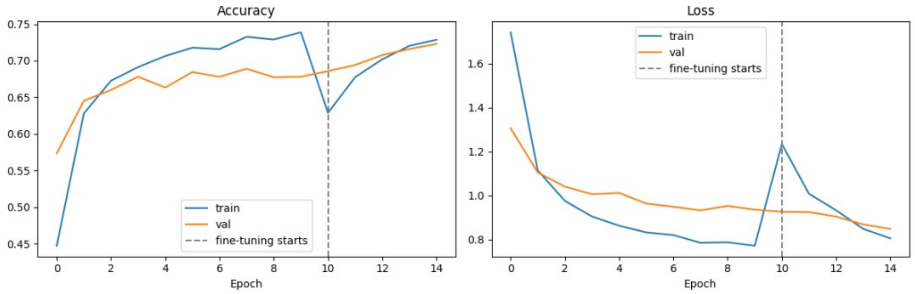


Figure 31 — Training curves showing accuracy and loss across both training phases; the dashed line marks where fine-tuning begins.

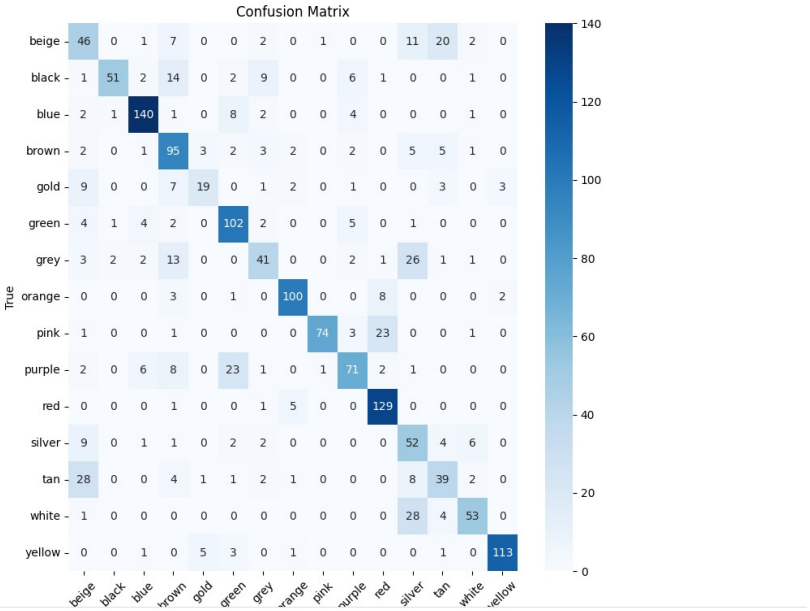


Figure 32 — Confusion matrix showing true versus predicted colour class across all 15 classes on the test set.

The trained model (car_color_model.keras) and its class names are loaded and cached at start-up. For each car box detected by MobileNet-SSD, the region is cropped, resized to 128×128, and passed through the classifier. If the predicted class is 'blue', a red bounding box is drawn; otherwise a blue box is used.

```
def get_dominant_color_label(crop_bgr):
    if crop_bgr.size == 0:
        return 'other'
    crop_rgb     = cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2RGB)
    crop_resized = cv2.resize(crop_rgb, (128, 128))
    crop_input   = np.expand_dims(crop_resized, axis=0).astype('float32')
    preds        = color_model.predict(crop_input, verbose=0)
    pred_class   = color_class_names[np.argmax(preds)]
```

```
    return 'blue' if pred_class.lower() == 'blue' else 'other'
```

5.7 Pedestrian Detection — HOG + SVM

Rather than train a custom pedestrian detector (which would need INRIA, CrowdHuman or CityPersons plus significant preprocessing and compute), a two-stage approach was used. MobileNet-SSD already detects 'person' as one of its 20 PASCAL VOC classes, giving a first pass of pedestrian detections as a by-product of the car-detection forward pass. A secondary HOG + SVM detector then runs as a backup to catch people the SSD misses at lower confidence, using OpenCV's built-in default people detector (pretrained on INRIA). This gives two independent systems with different characteristics — one deep-learning, one classical — whose detections are merged and deduplicated. HOG (Histogram of Oriented Gradients) divides the image into small cells, computes the distribution of gradient directions in each, and combines them into a descriptor that an SVM classifies; because it looks for fundamentally different features than the CNN, the two complement each other.

A key engineering improvement was a car-overlap filter for the HOG detections. Without it, HOG frequently mistakes car parts — front wheels, grilles, mirror assemblies — for people, since those regions share edge-gradient patterns with the human silhouette. The filter discards any HOG box where more than half its area overlaps a car box already detected by MobileNet-SSD.

```
# Discard HOG detections that are mostly inside a car bounding box
if any(contains_overlap(box, cb) for cb in final_car_boxes):
    continue      # likely a car part (wheel, grille, mirror), not a person

# Also discard if it heavily overlaps an SSD person detection
if any(boxes_overlap(box, pb[0]) for pb in final_person_boxes):
    continue      # already counted
```

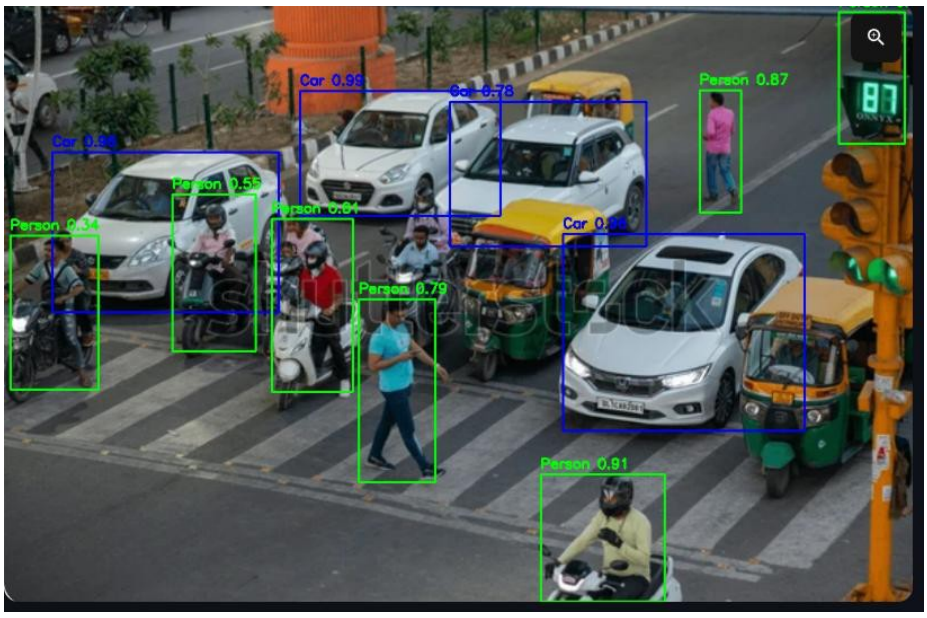


Figure 33 — People detection output — green bounding boxes on pedestrians in a real traffic scene.

5.8 The Streamlit Interface

The interface provides a JPG/PNG uploader, a two-column layout showing the original and annotated images side by side, three adjustable confidence sliders (car detection, person detection, and HOG backup sensitivity),

and a four-panel metric summary showing Total Cars, Blue Cars, Other Cars and People. The three sliders are a deliberate choice: different test images have different lighting, car sizes and crowd densities, so a fixed threshold that works on one image may miss detections or add false positives on another; exposing the thresholds as controls lets the user tune the system live rather than editing code.

```
car_conf   = st.slider('Car detection confidence threshold', 0.1, 0.9, 0.5, 0.05)
person_conf = st.slider('Person detection confidence (SSD)', 0.1, 0.9, 0.3, 0.05)
person_sensitivity = st.slider('HOG backup sensitivity', 0.0, 1.0, 0.5, 0.05)
uploaded_file = st.file_uploader('Upload an image', type=['jpg', 'jpeg', 'png'])

if uploaded_file is not None:
    # display input, run pipeline, display result, show metrics
```

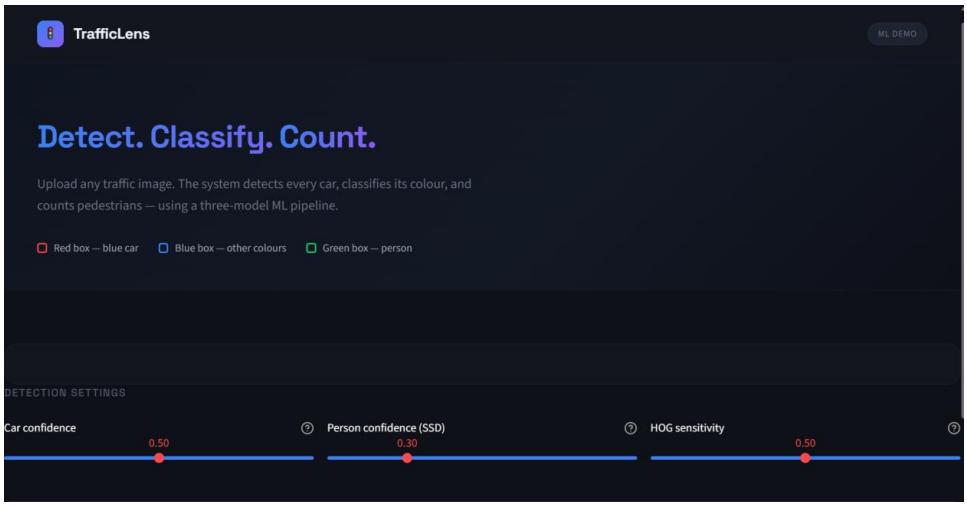


Figure 34 — TrafficLens interface — the upload area and the three interactive detection-confidence sliders.

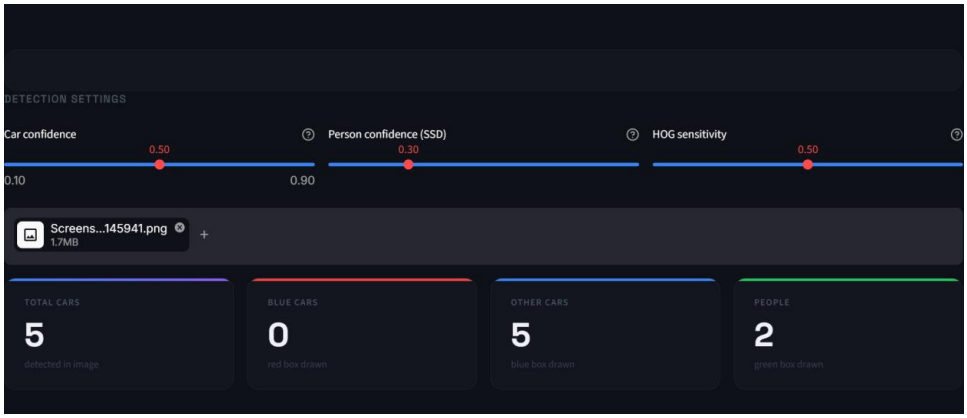


Figure 35 — The four-panel metric summary — total cars, blue cars, other cars and people counted for the current image.

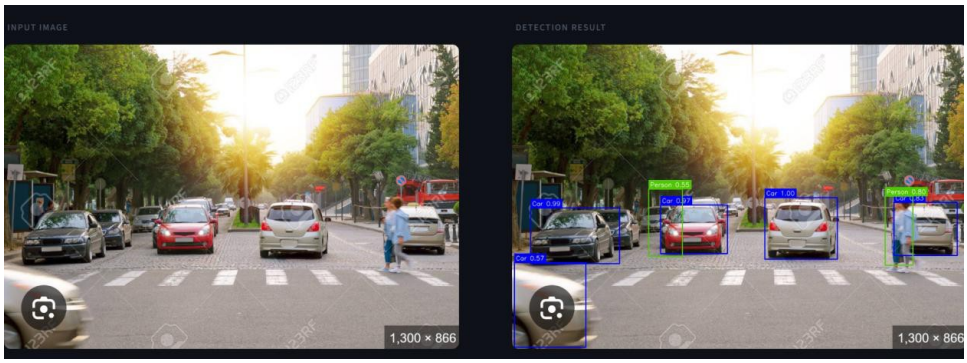


Figure 36 — Input image and annotated detection result shown side by side in the interface.

5.9 Challenges and Limitations

The YOLO restriction required a different architecture, met by MobileNet-SSD via OpenCV DNN — a genuine CNN detector covering the same classes, so the restriction produced a different system, not a weaker one. No suitable pre-labelled dataset existed for the exact detection task, which drove the decision to use a pretrained detector and turned out to be the right call. An early OpenCV Haar cascade (cars.xml) was rejected during testing because it produced numerous false positives on wheels, headlights and road markings and missed cars at non-frontal angles; switching to MobileNet-SSD resolved both immediately. Before the trained CNN, an HSV colour heuristic misclassified black and silver cars as blue under cool overcast lighting, because window glass and metallic paint reflect a cool sky hue; the trained MobileNetV2 classifier resolved most of these by learning saturation, texture and context rather than raw hue. The HOG detector's firing on car parts was fixed by the car-overlap suppression filter. Finally, most traffic images available online are steep aerial shots, AI illustrations, or watermarked stock — outside the eye-level distribution the models were trained on — which limited the range of usable test images.

The honest limitations are worth recording. Occluded and overlapping cars are frequently missed: when a large portion of a car's pattern is hidden, the visible fragment does not match what the detector learned, and dense bumper-to-bumper traffic can cause two cars to merge into one box or a gap to be detected as a spurious one — a fundamental limitation of single-stage detectors, not a tuning problem. Pedestrian detection is the weakest component, because MobileNet-SSD (PASCAL VOC) and HOG (INRIA) were both trained mostly on isolated, eye-level pedestrians, so the system reliably detects clearly visible upright people but misses or double-counts them in dense crowds and at steep angles, and sometimes flags signage or shadows as people. Both detectors can also fire on background structures that share visual characteristics with cars or people. Colour classification struggles on edge cases — very dark navy reads as black, metallic or pearlescent blue shifts toward grey or silver, and artificial orange or yellow streetlights skew perceived colour. And overall the pipeline performs best on eye-level or mildly elevated street-level photographs (roughly 10–30 degrees), degrading on pure overhead aerial images, heavily processed pictures and night-time photographs.

Component	Task	Result
MobileNet-SSD (Caffe, OpenCV DNN)	Car detection	0.95–1.00 confidence on clear eye-level images; degrades on occlusion and aerial angles
MobileNetV2 CNN (trained on VCoR)	Colour classification	Trained 15-class classifier; strong on clearly coloured cars in good lighting
MobileNet-SSD person + HOG/SVM	Pedestrian detection	Reliable on isolated upright pedestrians; inconsistent in crowds and aerial angles
Combined pipeline	Full annotation	Correct colour-coded box behaviour verified through

Component	Task	Result
		the GUI on multiple test images

This project shows that what appears to be a single detection task is actually a small system-design problem requiring three separate models in a coordinated pipeline: car detection, colour classification and pedestrian detection are independent sub-problems with different suitable models, different training data and different failure modes, and the real engineering is connecting them correctly. It also produced a practical lesson about pretrained models — their performance is largely determined by the distribution of their training data, performing well on eye-level, well-lit, unoccluded images and degrading predictably outside that distribution — and being able to articulate those limits is as important as maximising performance within them. Future improvements would extend the colour classifier to live video, swap the HOG backup for a COCO-trained SSD with a far richer pedestrian set, add a detector trained on elevated-camera footage, and build a curated dataset at the exact camera angle and lighting the system is expected to handle.

Part 6 — Sign-Language Detection System

Recognising five hand signs in real time — preprocessing, feature engineering and model selection. Baseline CNN vs. MediaPipe landmark model. Built with Streamlit.

6.1 Problem Statement

The task is to build a system that recognises sign language for a small set of chosen words, wired into a graphical interface supporting both image upload and a live webcam feed, and operational only within a fixed daily window — here, 6 PM to 10 PM. As with the other tasks, evaluation is based on overall behaviour and the interface rather than raw accuracy, which makes the data and engineering decisions as important as the model. Five signs were chosen — Hello, Yes, No, I Love You and Water — each with a clearly different hand shape so a model can tell them apart. This part walks through the data collection, preprocessing, feature engineering and model selection, and compares a baseline image model against the final landmark-based model the application uses.

6.2 Data Collection — Why a Custom Dataset Was Needed

The first instinct was to use an existing public sign-language dataset, but none fit a word-level image classifier for the five chosen signs, and confirming this took real effort. The popular ASL datasets on Kaggle contain only the alphabet — one folder per letter — with no whole-word signs such as Hello or Water. The few word-level datasets that exist were the wrong shape: large collections such as WLASL are videos rather than still images, and others were supplied as object-detection data with bounding boxes (YOLO format) rather than simple class folders, which did not suit a straightforward image classifier and was not permitted here. Research datasets that did include word gestures kept those words only as videos, while their still-image portion was again just the alphabet.

Because clean image-classification data for specific whole-word signs essentially does not exist off the shelf, and the brief allows "words of your choice," the dataset was built first-hand with a webcam. A capture script showed each target sign on screen and recorded samples, and the data was collected across several rounds with the background, lighting and position deliberately changed between rounds. Because every sign was recorded in every environment, no single background could be tied to one sign — forcing any model to learn the hand rather than the room.

```
WORDS = ['Hello', 'Yes', 'No', 'ILoveYou', 'Water']
cap = cv2.VideoCapture(0)

for word in WORDS:
    save_dir = os.path.join('data/train', word)
    os.makedirs(save_dir, exist_ok=True)
    count = len(os.listdir(save_dir))        # continue numbering across rounds
    while capturing:
        ok, frame = cap.read()
        frame = cv2.flip(frame, 1)
        cv2.imwrite(f'{save_dir}/img_{count:04d}.jpg', frame)
        count += 1
```

I recorded the whole dataset myself — every sample is my own hand performing each sign — capturing around 400 samples per sign for a balanced set of roughly 2,000 images across the five classes. That detail matters for what happened next: because it was all one person in a handful of settings, the raw-image model had very little real variety to learn from, which is exactly where the first approach came unstuck.

6.3 Data Preprocessing

Two different preprocessing pipelines were used, one per modelling approach. Keeping them separate and correct mattered, because feeding a model inputs prepared differently from how it was trained silently produces wrong predictions. For the baseline image model, every frame was resized to 64×64 pixels, kept in three colour channels, and scaled to the 0–1 range, loaded with a shuffled split so training and validation each contained a fair mix of all five signs and all backgrounds. Light augmentation — small rotations, zooms and translations — was applied to the training images only.

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.2, subset='training',
    seed=42, image_size=(64, 64), batch_size=32)      # shuffles before split

normalize = layers.Rescaling(1./255)                   # pixels -> 0..1
augment = tf.keras.Sequential([
    layers.RandomRotation(0.05), layers.RandomZoom(0.1),
    layers.RandomTranslation(0.1, 0.1)])               # training only
```

For the final model, preprocessing means running MediaPipe on each frame to detect the hand and return the coordinates of its 21 joints. No pixels are kept — only the joint positions move forward, and if MediaPipe finds no hand, the frame is skipped rather than guessed at.

```
res = hands.process(rgb_frame)
if res.multi_hand_landmarks:
    hl = res.multi_hand_landmarks[0]
    coords = [(lm.x, lm.y) for lm in hl.landmark]      # 21 (x, y) joints
```

6.4 Feature Engineering

This is where the two approaches differ most, and where the project's key idea lives. The image model performs almost no feature engineering — the raw pixels are handed to the network, which is left to discover useful features on its own, and (as the next section shows) it discovered the wrong ones. The landmark model, by contrast, engineers a compact, meaningful feature vector by hand. The 21 joint coordinates are normalised in two steps so the features describe only the shape of the hand, independent of where it is in the frame or how big it appears: translation invariance subtracts the wrist joint from every point, moving the hand to a common origin so its position no longer matters; scale invariance divides the coordinates by the hand's overall size, so distance from the camera no longer matters. The result is a 42-value vector (21 joints × x and y) that captures the finger configuration and nothing else — the single engineering step that lets the final model generalise to a live camera.

```
def normalize(landmarks):
    pts = np.array(landmarks, dtype=np.float32)   # 21 (x, y) points
    pts = pts - pts[0]                             # 1) wrist to origin (translation)
    m = np.max(np.linalg.norm(pts, axis=1))
    if m > 0:
        pts = pts / m                             # 2) scale by hand size (scale)
    return pts.flatten()                          # 42 shape-only features
```

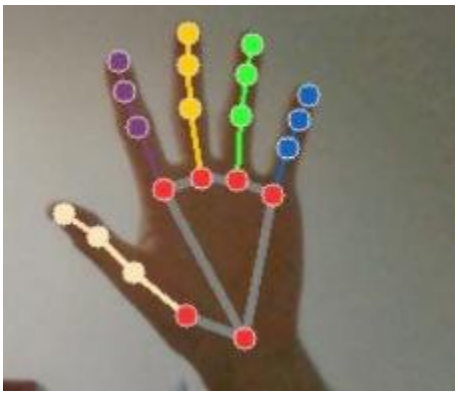


Figure 37 — MediaPipe's 21 hand landmarks drawn on a hand — the joint skeleton the model classifies, with pixels discarded.

6.5 Model Selection and Comparison

A Convolutional Neural Network was the natural baseline, because CNNs are the standard tool for image classification. The network used three convolutional blocks feeding a dense classifier with five softmax outputs, trained on Kaggle with a GPU.

```
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', padding='same',
                  input_shape=(64,64,3)),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Dropout(0.25),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Dropout(0.25),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(256, activation='relu'), layers.Dropout(0.5),
    layers.Dense(5, activation='softmax')])
```

I trained this CNN first, on my own recorded images. On the validation set it reached around ninety percent accuracy and — once an early data-split bug was fixed — spread its predictions across all five signs, so on paper it looked finished. In the live application, though, it collapsed, predicting a single sign for almost everything. The problem was heavy overfitting, and it came straight from the data being all me: because the entire dataset was one person performing the signs in a few fixed settings, and because consecutive captured frames are nearly identical, near-duplicate images ended up on both sides of the train/validation split and inflated the score. The model had really memorised me, the recording sessions and their backgrounds rather than the hand shapes themselves — so a live frame with different lighting and a fresh background looked unfamiliar, and it defaulted to one class. That failure is what pushed me to abandon the raw-image approach and move on to MediaPipe.

Because the failure came from the representation rather than the architecture, the advanced model changes what the model looks at. MediaPipe extracts the hand's 21 joints, the normalisation above turns them into 42 shape-only features, and a small Multi-Layer Perceptron classifies those features into the five signs. A CNN is deliberately not used here — convolutions exist to scan pixels, and there are no pixels, only a short list of numbers.

```
from sklearn.neural_network import MLPClassifier
clf = MLPClassifier(hidden_layer_sizes=(128, 64),
                    max_iter=600, random_state=42)
clf.fit(X_train, y_train)          # X = 42 landmark features per sample
```

Training runs locally in seconds with no GPU, since the input is a table of coordinates. The model reached roughly ninety percent on its held-out test set — but unlike the CNN, this figure held up live, because background and lighting never reach the model.

Aspect	Baseline — CNN on images	Advanced — MLP on landmarks
Input	64×64 RGB pixels	42 normalised landmark values
Feature engineering	None (learned from pixels)	Translation + scale-invariant geometry
Framework	TensorFlow / Keras	MediaPipe + scikit-learn
Hardware	GPU (Kaggle)	Laptop CPU, trains in seconds
Validation accuracy	≈ 90%	≈ 90%
Live webcam	Collapsed to one class	Works reliably
Why	Memorised background	Sees only hand shape

The two models score almost the same on validation, but only the landmark model survives live use — a reminder that validation accuracy alone can be misleading when the data contains near-duplicates, and that the right representation matters more than the raw number.

6.6 Why the Camera Could Not Focus on the Hand

A core difficulty with the image approach is that a webcam does not see a hand — it sees a whole scene, with the face, upper body, background and room lighting, and the hand occupying only a small part of it. The CNN was given that entire frame and left to work out which part mattered, and with limited, self-recorded data it latched onto the easiest signal available: the background and overall appearance of each recording session. Nothing in the pipeline told it to concentrate on the hand. MediaPipe solves exactly this: it is a dedicated hand-tracking model that locates the hand within the scene and returns only its joint positions, discarding everything else. In effect it does the focusing that the camera and the CNN could not — the hand is found explicitly, rather than hoped for implicitly. This is the single biggest reason the project moved from a plain CNN on full frames to MediaPipe landmarks.

6.7 Visualizations and Insights

A bar chart of samples per sign confirms the dataset is balanced, with roughly 400 images each. Balance matters because an imbalanced set would let a model score well simply by favouring the largest class.

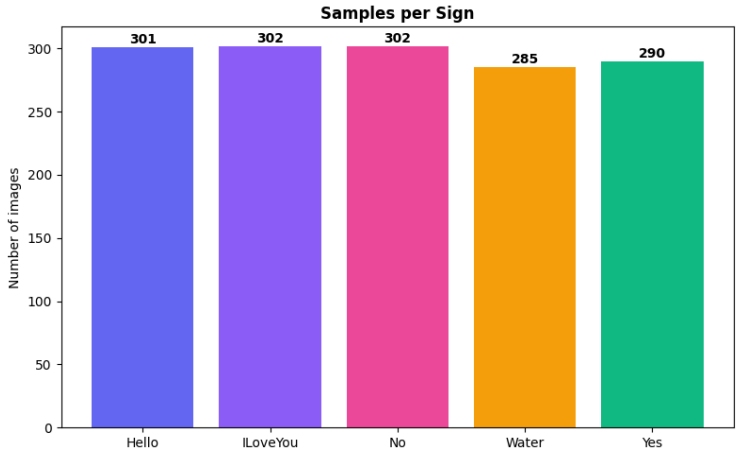


Figure 38 — Samples per sign — a balanced dataset of roughly 300 usable images per class across the five signs.

A grid showing one captured frame per sign documents the five distinct hand shapes and the varied backgrounds used during capture, making the diversity of the dataset visible at a glance.

Figure 39 — One example frame per sign (Hello, I Love You, No, Water, Yes), showing the distinct hand shapes and varied backgrounds.

A per-class accuracy chart shows a consistent pattern across runs: the visually distinct signs (Hello, Water, I Love You) score highest, while the two most compact hand shapes — Yes and No — are the hardest, since a fist and a two-finger pinch look similar at low resolution. A confusion-matrix heatmap confirms this: a bright diagonal indicates mostly-correct predictions, and the main off-diagonal cell sits between Yes and No.

```
import seaborn as sns
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(all_true, all_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
```

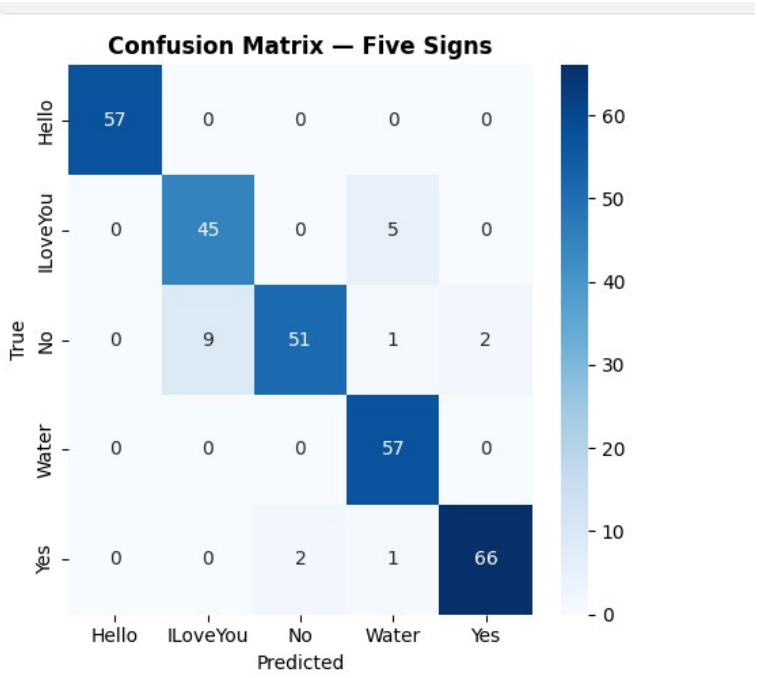


Figure 40 — Confusion matrix for the five signs — a bright diagonal, with the main confusion between Yes and No as expected.

6.8 The Interface and Time Restriction

The interface was built with Streamlit and offers two modes. In upload mode, a photo is processed, the hand skeleton is drawn on it, and the predicted sign, confidence and a probability bar chart are shown. In live mode, a toggle opens the webcam and the detect → normalise → classify loop runs on each frame, drawing the skeleton and prediction onto the video; predictions are smoothed over a short rolling window so a single noisy frame cannot make the label flicker. The 6–10 PM restriction is enforced by a small time-check that runs before any detection feature is shown: inside the window the full interface is unlocked; outside it, the tools are replaced by a lock screen stating when the app reopens.

```
START_HOUR, END_HOUR = 18, 22          # 6 PM to 10 PM
def time_open():
    return START_HOUR <= datetime.now().hour < END_HOUR
```

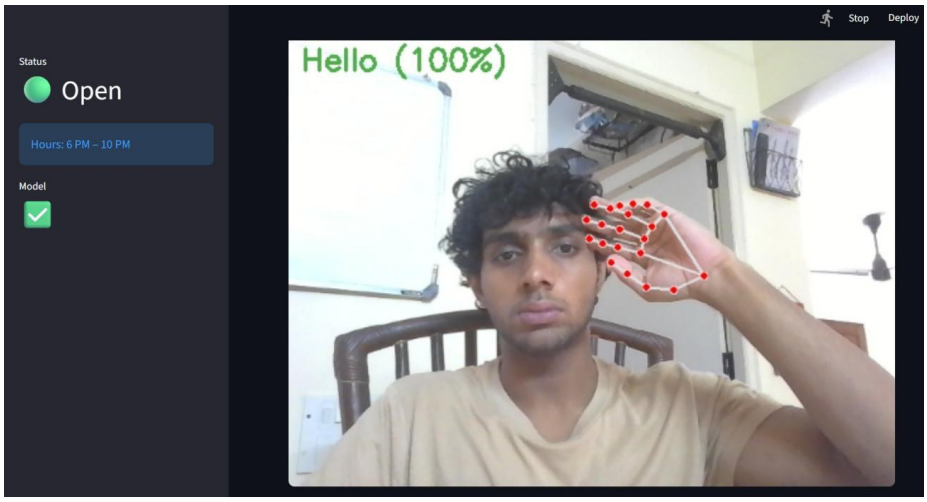


Figure 41 — Live mode inside the open window — the hand skeleton drawn on the webcam feed with the prediction "Hello (100%)".

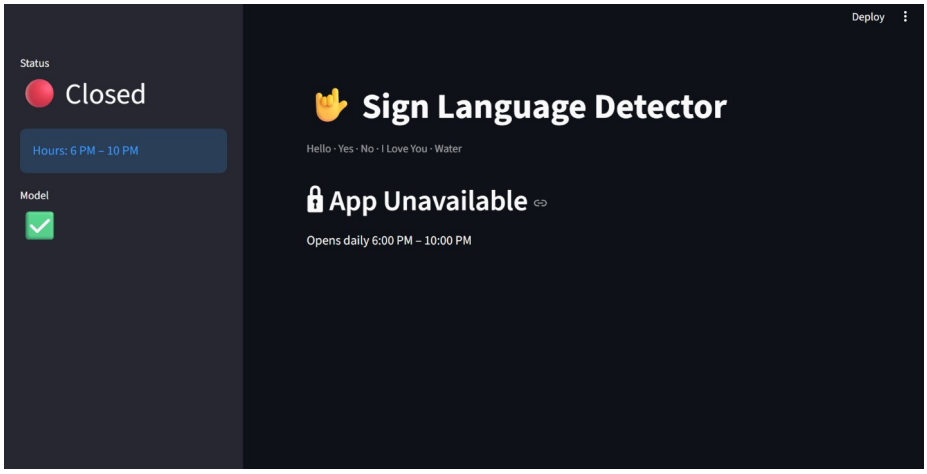


Figure 42 — Outside the 6–10 PM window the detection tools are replaced by a lock screen stating when the app reopens.

6.9 Results, Limitations and Conclusion

Stage	Choice	Result
Data collection	Custom webcam capture, multi-background	Balanced ~400 samples per sign
Preprocessing	Landmark extraction via MediaPipe	Hand isolated from the scene
Feature engineering	Translation + scale normalisation	42 shape-only features
Baseline model	CNN on images	≈ 90% validation, failed live
Final model	MLP on landmarks	≈ 90% test, works live
Interface	Streamlit, upload + live, 6–10 PM gate	All requirements met

The decisive outcome is qualitative as much as numerical: where the baseline CNN collapsed to a single prediction on the live webcam, the landmark model recognises all five signs in real time under everyday conditions, with Yes and No the only pair that occasionally trades places. A few honest limitations remain. The data was captured from one person's hands in a handful of sessions, so the model learns that individual's hand proportions and may be less accurate for others — a robust system would need many signers. Only five static

signs are supported, each treated as a single hand pose, while real sign language is far larger and many signs involve motion a one-frame classifier cannot capture. And the whole pipeline relies on MediaPipe finding the hand: if the hand is poorly lit, partly out of frame, turned side-on, or moving quickly, MediaPipe may fail and the model cannot predict at all.

The clearest lesson of this part is that the hardest part of a sign-recognition system is often the data and the representation, not the network. A baseline CNN on raw images appeared to work at around ninety percent validation accuracy yet failed live because it memorised backgrounds and near-duplicate frames rather than the signs. The working system replaces pixels with geometry — MediaPipe locates the hand and extracts 21 joints, a normalisation step removes position and scale so only the hand shape remains, and a small network classifies that shape — so that background and lighting never reach the model and live performance finally matches training. Future improvements would expand the vocabulary, collect data from several people, incorporate both hands and motion for signs that depend on movement, and add a confidence-based “no sign detected” state so the system stays silent until it is genuinely sure.

Closing — Common Threads Across the Six Systems

Taken together, the six systems tell a more consistent story than any one of them does alone. Each began as a one-line brief that hid several sub-problems, and in every case the first and most valuable move was the same: decompose the task, choose the right tool for each piece, and reduce the actual requirement to a short, readable decision function. In Part 1 that function flips gender by hair length inside an age band; in Part 3 it routes to conditional attributes by ethnicity; in Part 4 it gates age and emotion behind a gender check; in Part 5 it turns a colour label into a red or blue box. The models exist to supply the inputs; the logic that satisfies the brief is deliberately small and easy to verify by hand — which is exactly what briefs that weight behaviour over accuracy reward.

A second thread is honesty about what is original and what is borrowed. Across the six projects, some components are trained from scratch — the age/gender CNN, the hair ResNet18, the ethnicity CNN, the colour MobileNetV2, the classical voice pipeline — while others are integrated: DeepFace for emotion, wav2vec2 for deep voice analysis, MobileNet-SSD for detection, MediaPipe for hand tracking. Each report states which is which, and treats accurately drawing that line as part of the engineering. The same discipline appears in reusing work: the Part 1 age/gender model is carried unchanged into Parts 2 and 3, and Part 1's future-work face-detection step becomes Part 2's opening stage, so the body of work compounds rather than repeating itself.

A third thread is data quality and representation. The biggest failures in the whole report were not architectural — they were about data. CelebA's texture labels taught the hair model a confidently wrong rule; near-duplicate frames inflated the sign CNN's score until it collapsed live; thin elderly-voice data capped the age models regardless of whether they were a Random Forest or a transformer; and pretrained detectors degraded predictably the moment test images left their training distribution. The recurring fix was to change the data or the representation rather than to keep tuning the model — manual labelling on trusted data, MediaPipe landmarks instead of raw pixels, per-class sampling, and pretrained models used within the distribution they were built for.

Finally, every system is delivered as a working Streamlit application and every part ends by naming its limitations plainly. That combination — something that runs end to end, described together with an honest account of where it breaks — is the through-line of the internship. The individual accuracies matter, but the more durable result is the demonstrated ability to take an ambiguous problem apart, assemble a pipeline from the right mix of built and integrated parts, and be clear-eyed about what the finished system can and cannot do.